



Tema 18: Deep Learning

Contenidos de la sesión

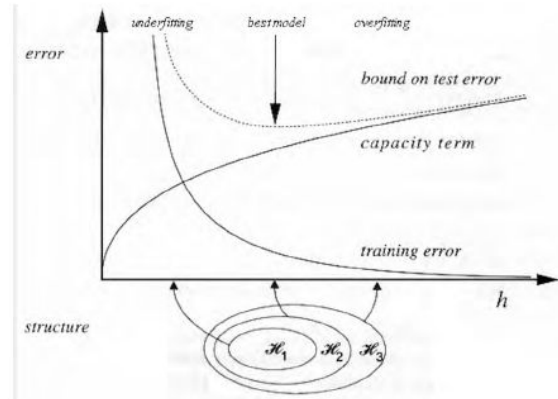
- Introducción
- Historia
- El perceptrón
- Redes Neuronales
- Deep Learning
- Entrenamiento y técnicas



Introducción

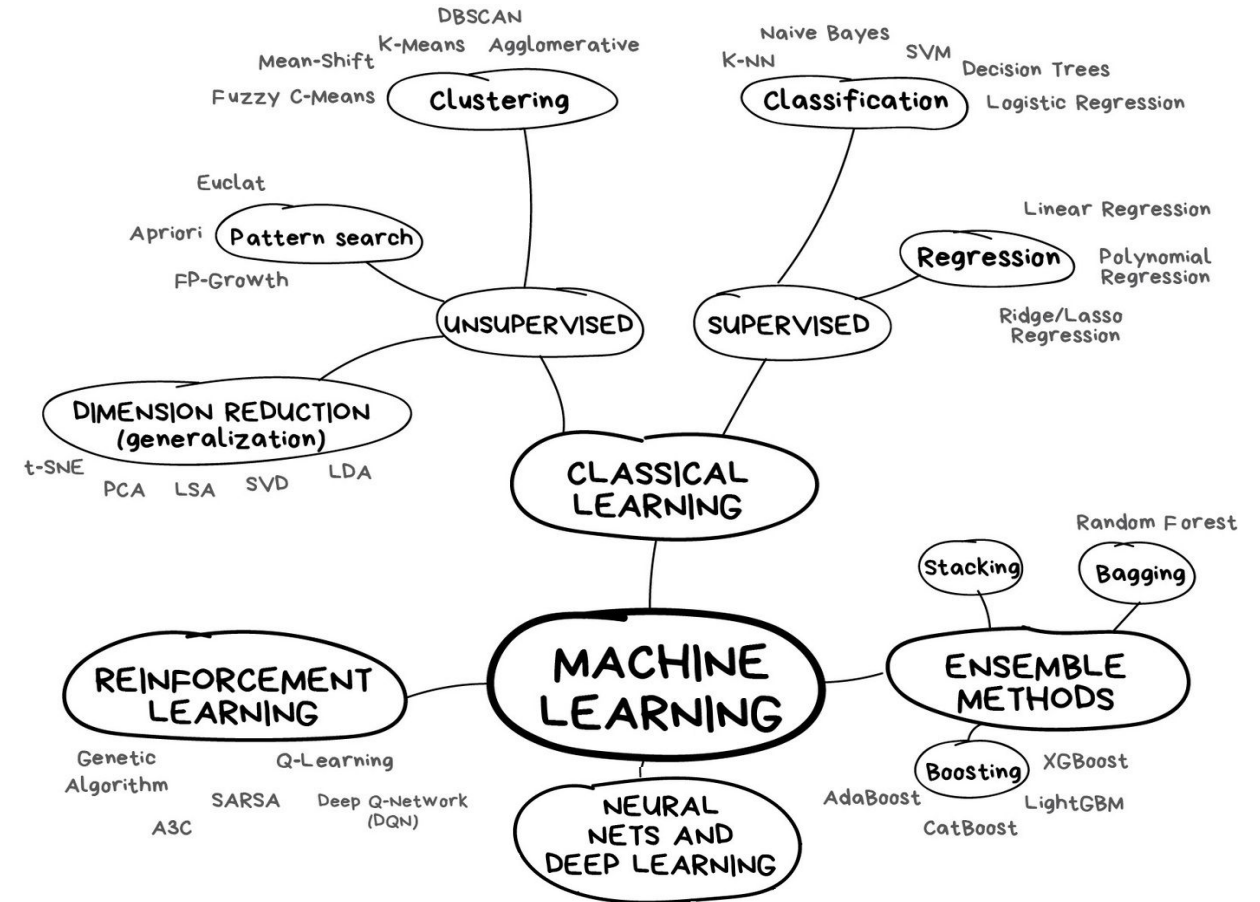


Learning

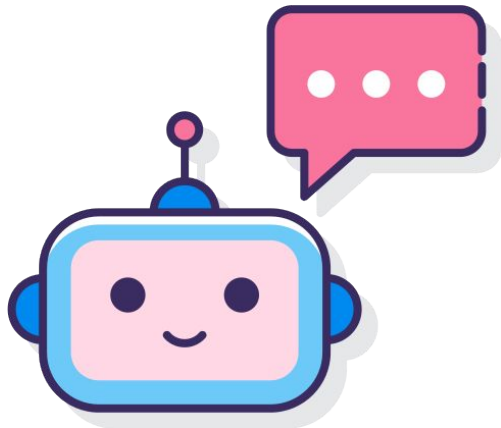


M. Sewell (2008). Structural risk minimization. University College London.

A (machine) **learning** problem consists of setting up, tuning, or **finding a configuration** for modifiable components of an (intelligent or learning) **system** responsible for its success or failure.



Asistentes virtuales y chatbots



Casos reales:



- **Chatbots:** modelos de lenguaje basados en Transformers.



- **Asistentes virtuales:** reconocen voz, entienden lenguaje natural y generan respuestas

Medicina



Casos reales:



- **Generación de proteínas: Alphafold** predice la estructura de proteínas en minutos (anteriormente se tardaban meses/años).



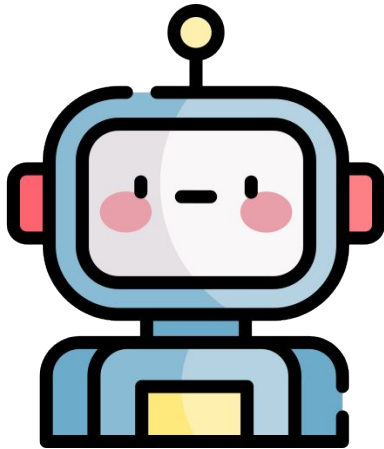
- **Detección en imágenes:** detectar enfermedades a partir de imágenes médicas.



Google Health

- **Seguimiento de pacientes:** encontrar patrones en datos recogidos de manera no invasiva.

Robótica



Casos reales:



- **Robots quirúrgicos** (Da Vinci): aunque su control principal es humano, se están incorporando modelos de IA para asistir en tareas como segmentación de órganos y planificación quirúrgica.

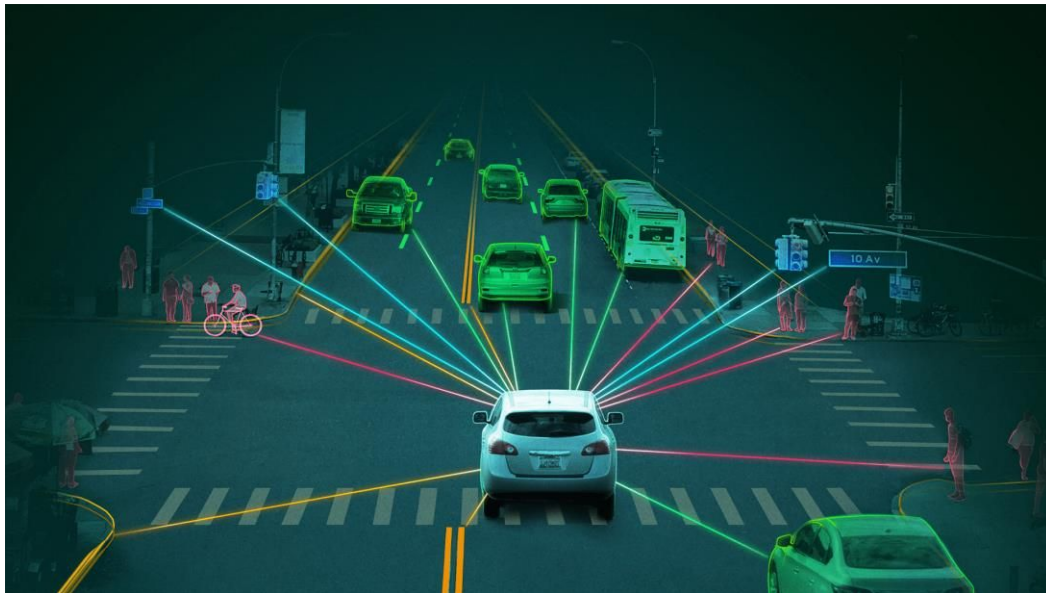


- **Robótica doméstica:** Aspiradoras inteligentes que combinan sensores y deep learning para mapear entornos, evitar obstáculos y aprender de los hábitos del usuario.



- **Robótica industrial:** movimiento de productos, montaje de piezas, etc. mediante visión artificial.

Conducción autónoma



Casos reales:



- **Coches:** redes neuronales para interpretar cámaras, radares y tomar decisiones en tiempo real.



- **Aviación:** despegues y aterrizajes automáticos usando visión por computadora.



- **Drones:** deep learning para navegación y seguimiento de objetos en tiempo real.

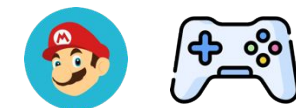
Entretenimiento



Casos reales:

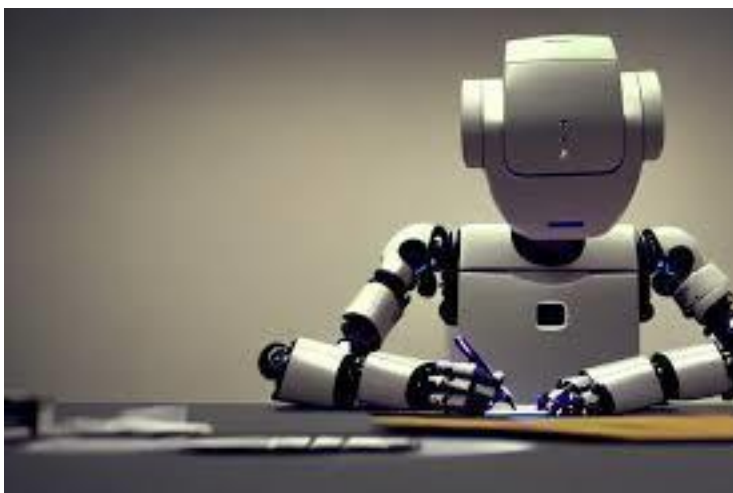


- **Sistemas de recomendación:** deep learning para recomendar películas y series basándose en el comportamiento del usuario y características del contenido.



- **Videojuegos:** redes neuronales para reconstruir imágenes en alta resolución, generación de personajes y escenarios.

Creación de contenido



Casos reales:



- **Generación de imágenes** artísticas a partir de texto



- **Generación de vídeos** a partir de texto.

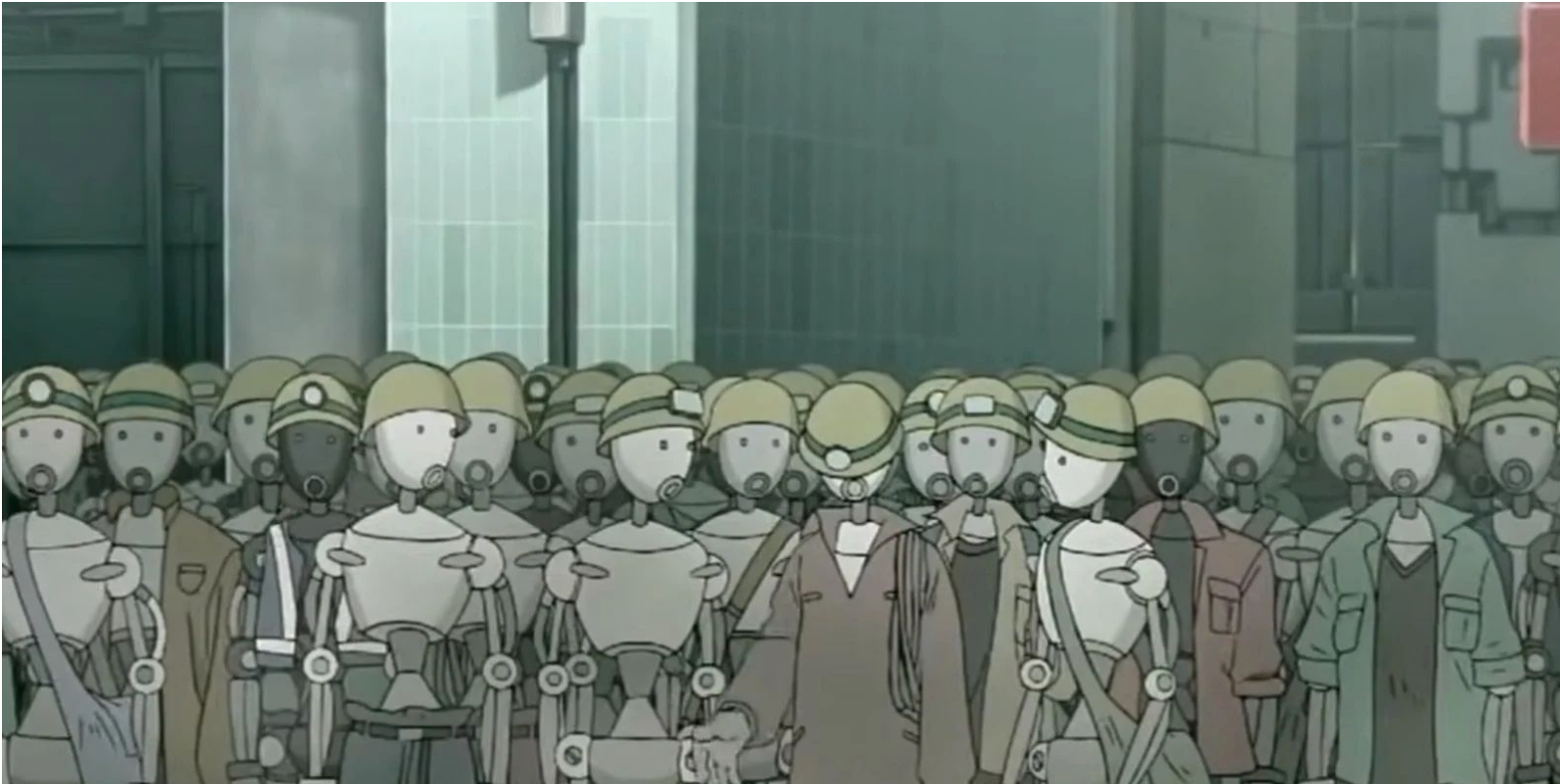


- **Generación de música** a partir de texto.

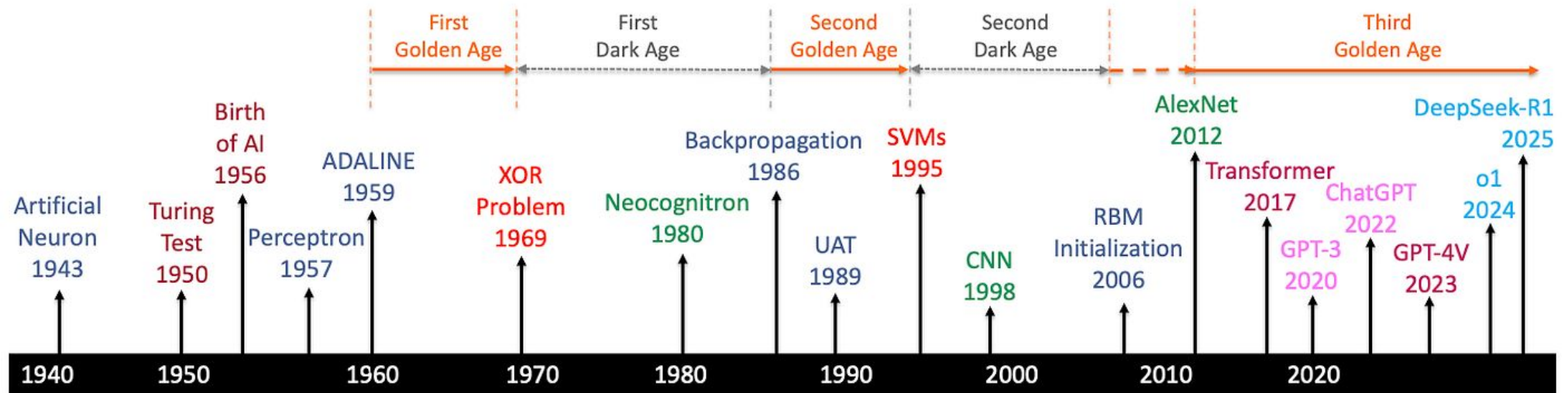
Historia



Una breve historia de Deep Learning



A Brief History of AI with Deep Learning



McCulloch-Pitts

Rosenblatt

Widrow-Hoff

Minsky-Papert

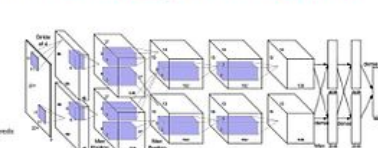
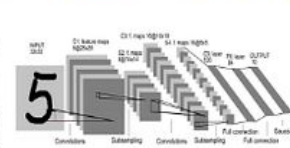
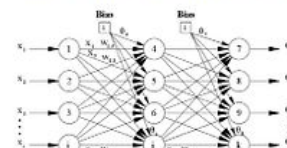
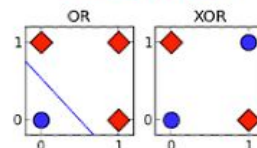
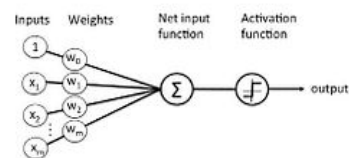
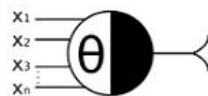
Rumelhart, Hinton et al.

LeCun

Hinton-Ruslan

Krizhevsky et al.

Vaswani



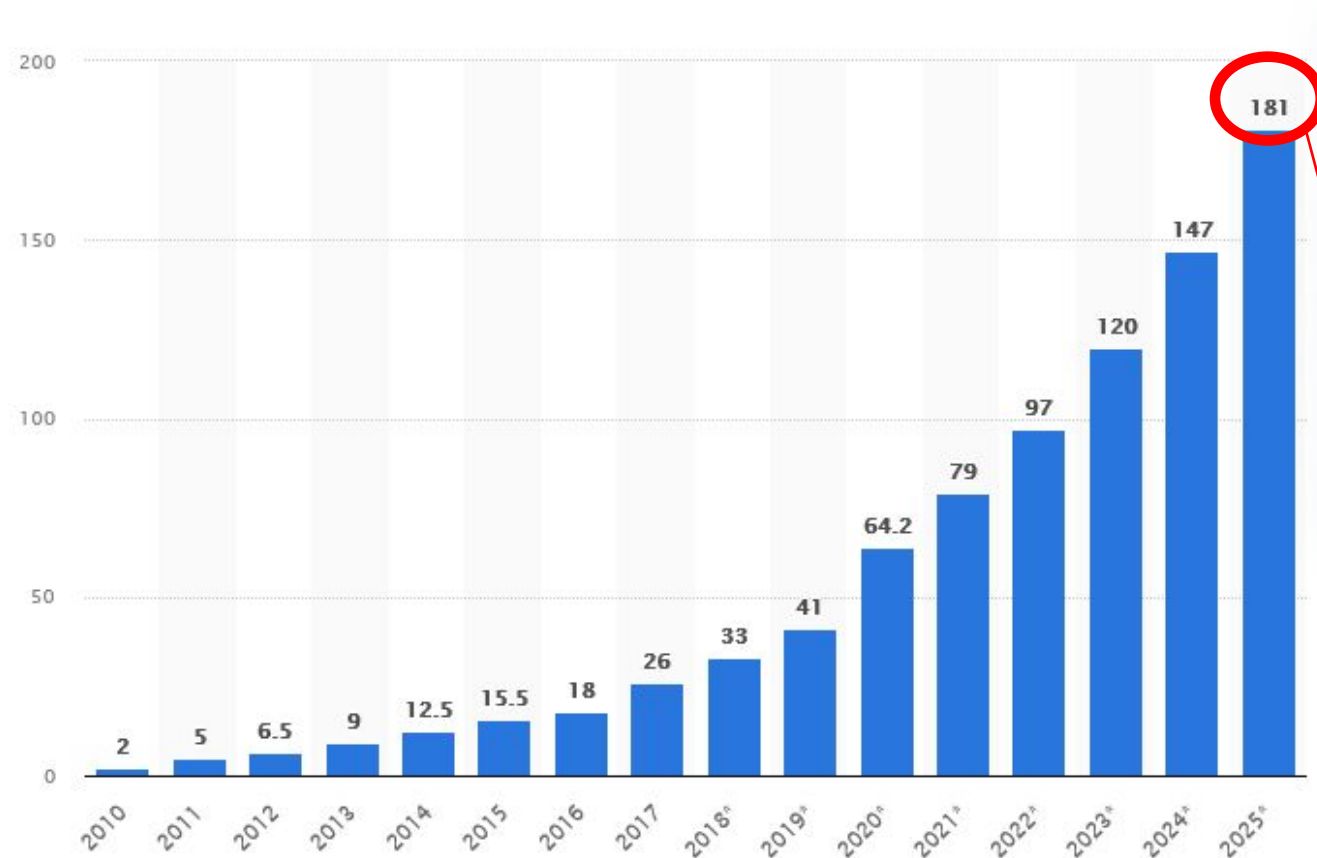
Una breve historia de Deep Learning

Los datos - muy importantes



La cantidad de información aumenta - tenemos datos para entrenar IAs

Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2020, with forecasts from 2021 to 2025

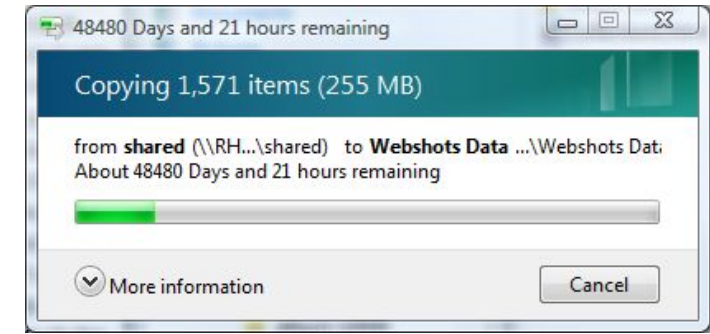
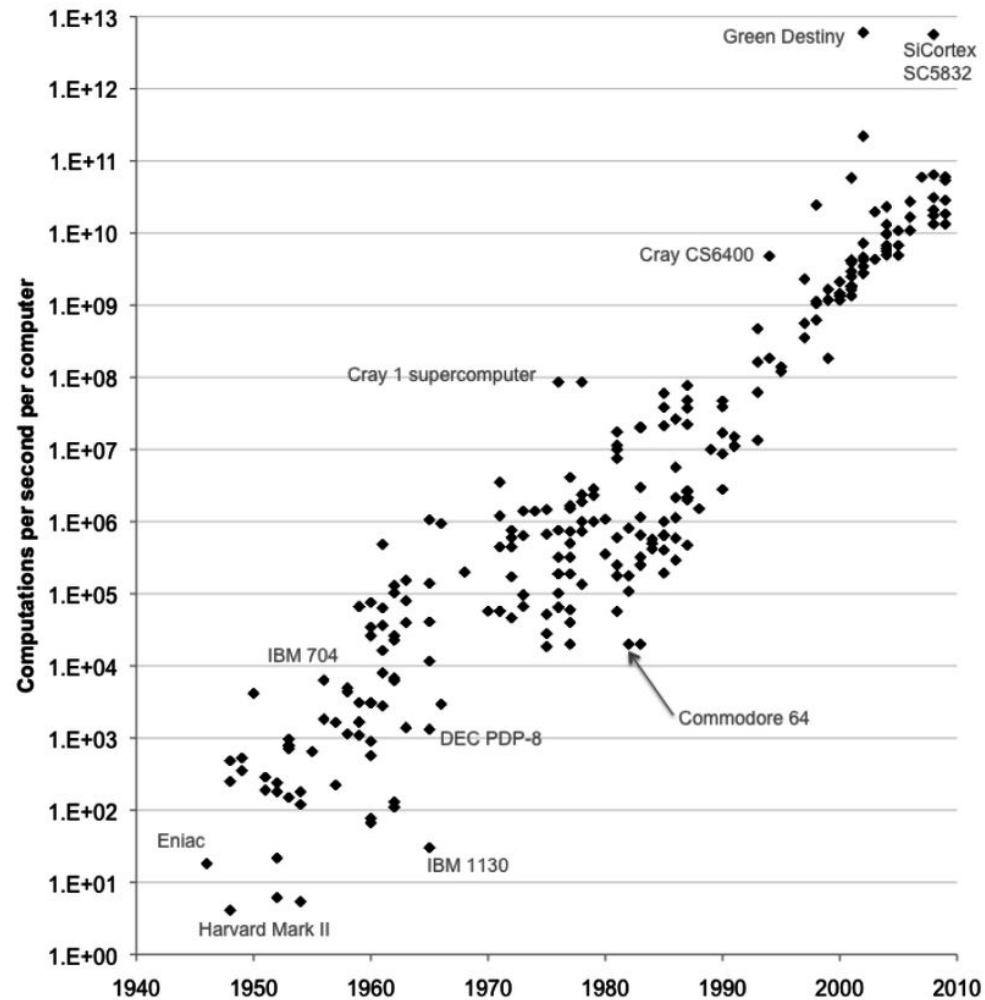


Multiples of Bytes		
Unit (Symbol)	Value (SI)	Value (Binary)
Kilobyte (kB)	10^3	2^{10}
Megabyte (MB)	10^6	2^{20}
Gigabyte (GB)	10^9	2^{30}
Terabyte (TB)	10^{12}	2^{40}
Petabyte (PB)	10^{15}	2^{50}
Exabyte (EB)	10^{18}	2^{60}
Zettabyte (ZB)	10^{21}	2^{70}
Yottabyte (YB)	10^{24}	2^{80}

10¹⁹fotos de gatos (en jpg)
10000000000000000000 gatos

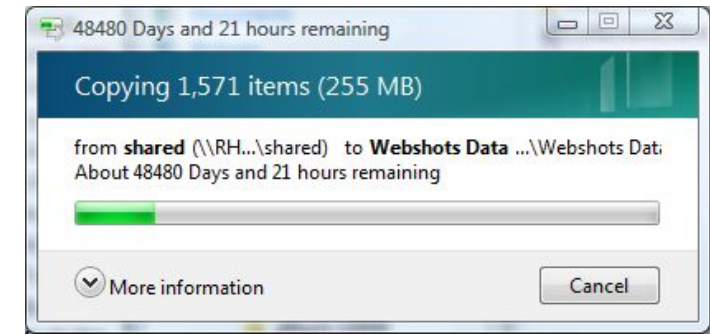
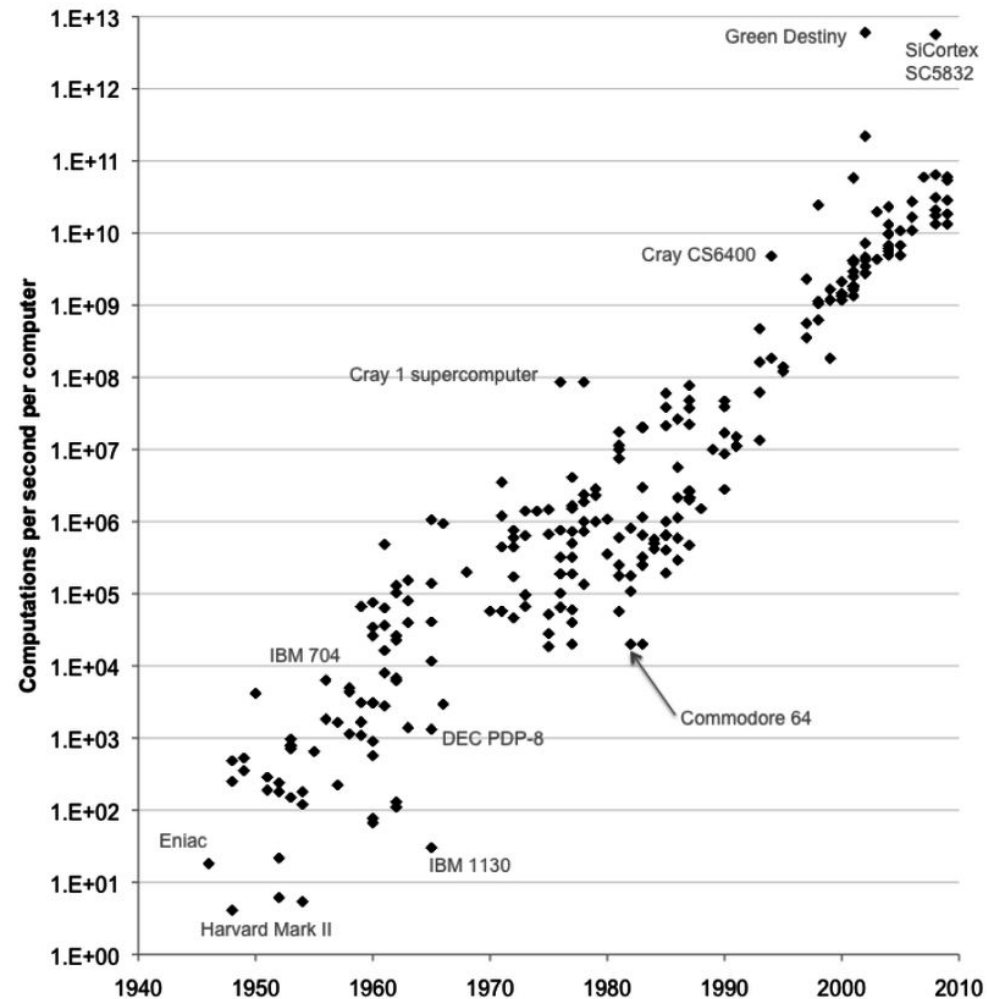
Una breve historia de la IA

El poder de cómputo aumenta - podemos entrenar esas IAs



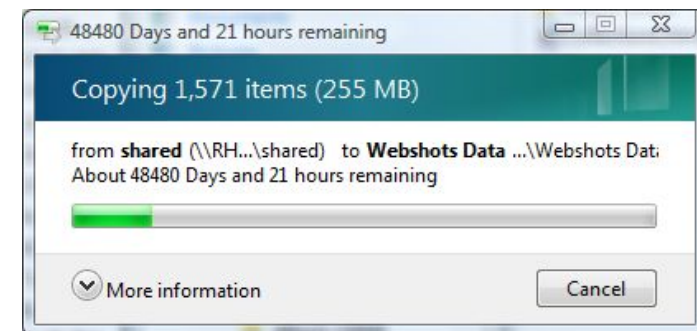
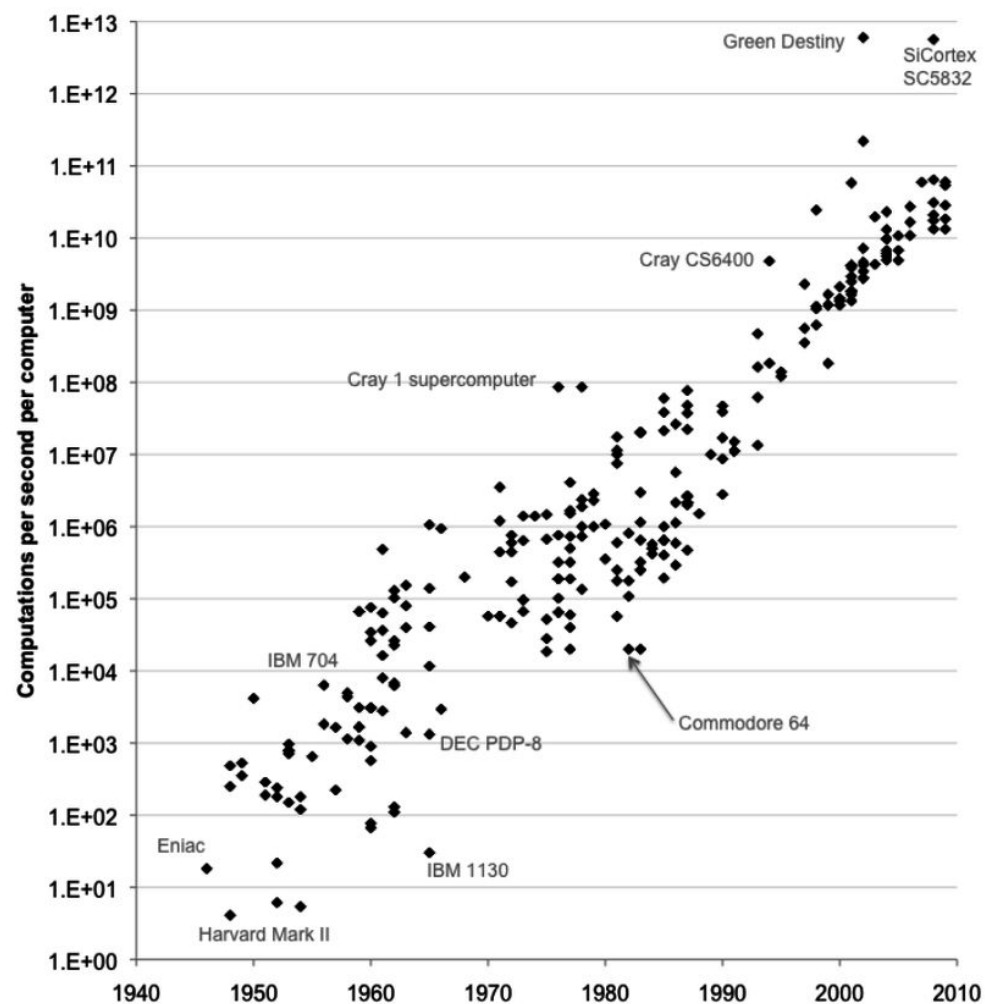
Una breve historia de la IA

El poder de cómputo aumenta - podemos entrenar esas IAs



Una breve historia de la IA

El poder de cómputo aumenta - podemos entrenar esas IAs



Una breve historia de la IA

Las técnicas mejoran, podemos aprender de los datos

Las redes de neuronas

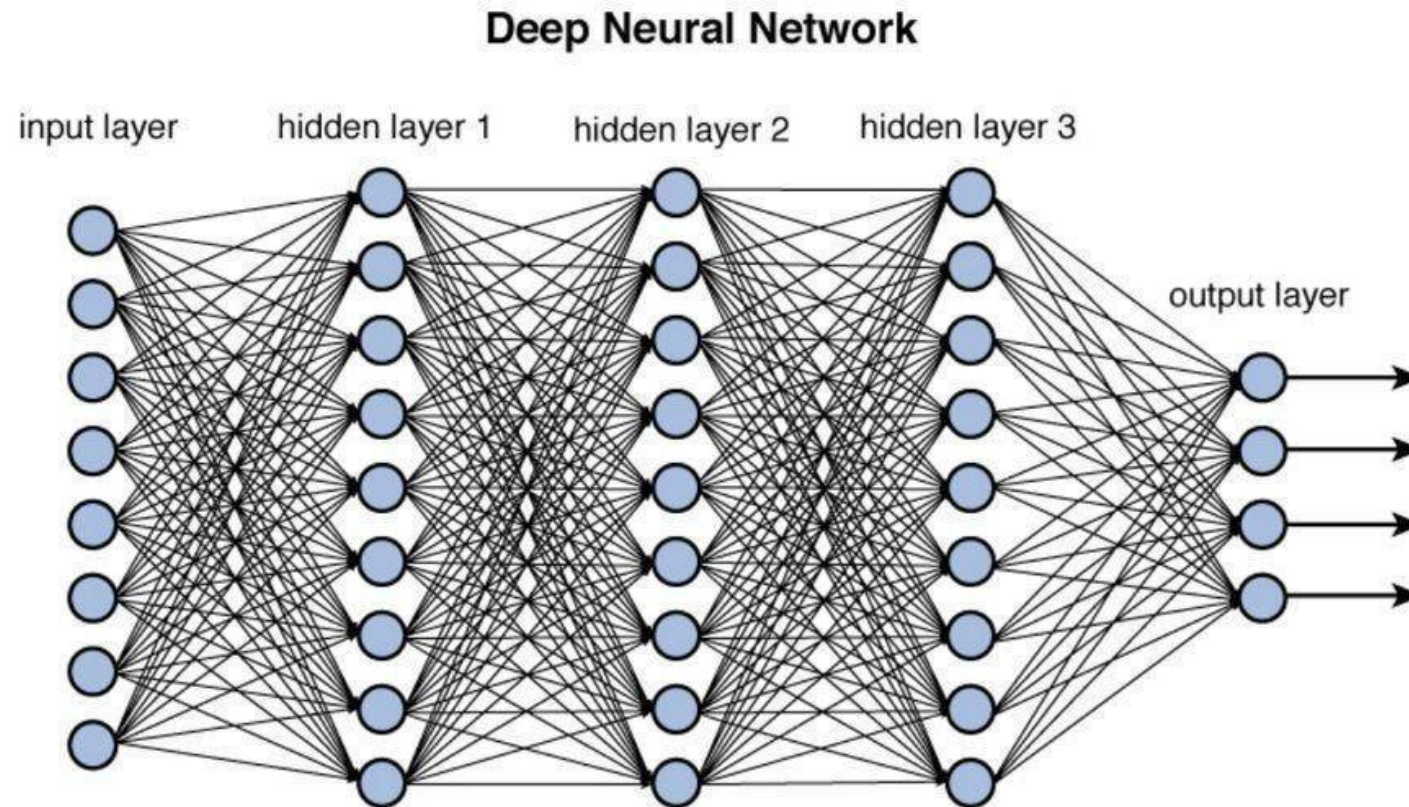
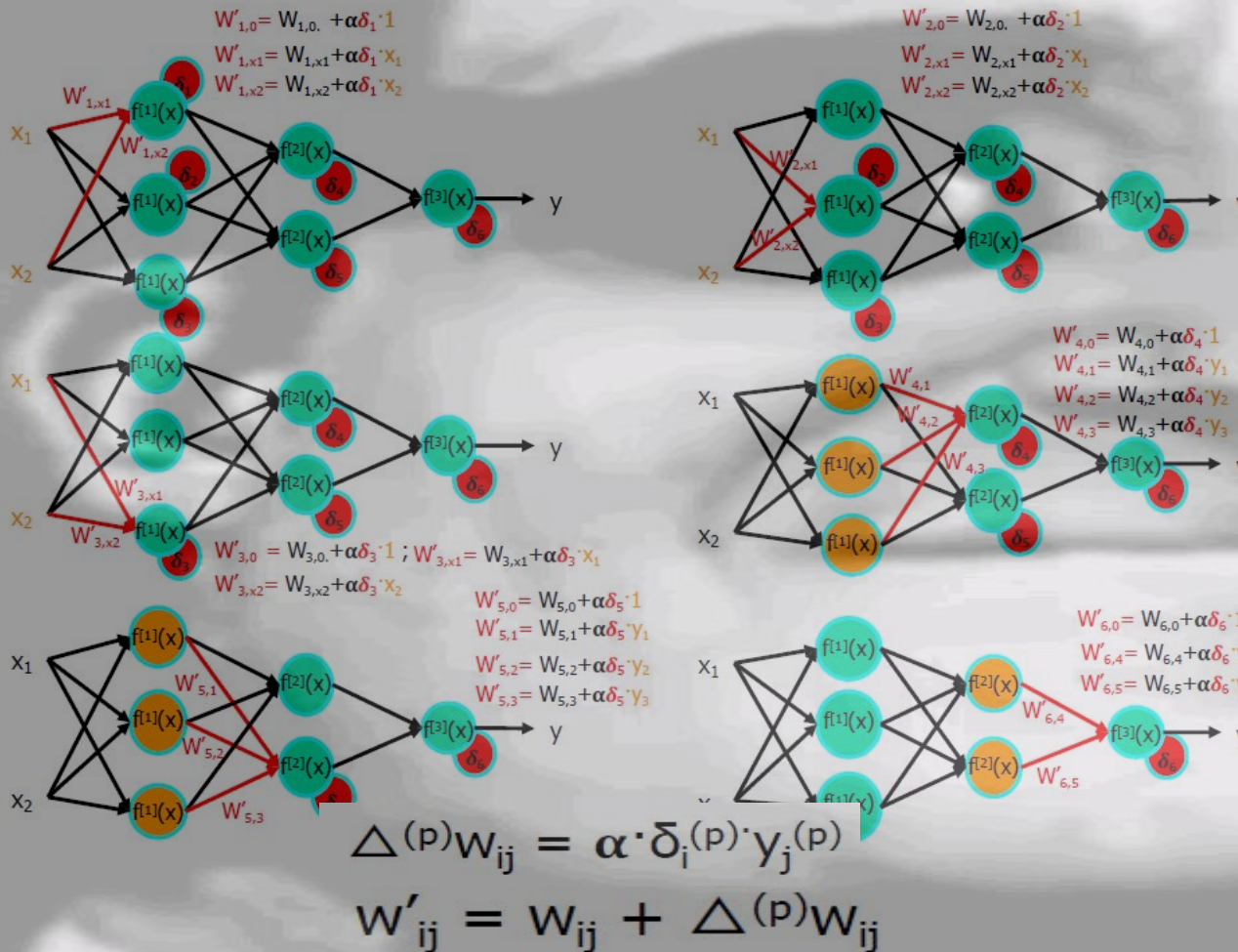


Figure 12.2 Deep network architecture with multiple layers.

Una breve historia de la IA

Las técnicas mejoran, podemos aprender de los datos

Por qué funcionan?



(a) compute error on g_j

$$\frac{\partial E}{\partial g_j} = \sum_i \underbrace{\sigma'(h_i)}_{\text{how } h_i \text{ will change as } g_j \text{ changes}} v_{ij} \underbrace{\frac{\partial E}{\partial h_i}}_{\text{was } h_i \text{ too high or too low?}}$$

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \frac{\partial E}{\partial g_j} \underbrace{\sigma'(g_j)}_{\text{do we want } g_j \text{ to be higher/lower}} f_k \underbrace{\frac{\partial f_k}{\partial u_{jk}}}_{\text{how } g_j \text{ will change if } u_{jk} \text{ is higher/lower}}$$

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

The vanishing gradient problem - Details

- Similar to backprop but less efficient formulation
- Useful for analysis we'll look at:

$$\frac{\partial E_t}{\partial W} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \boxed{\frac{\partial h_t}{\partial h_k}} \frac{\partial h_k}{\partial W}$$

- Remember: $h_t = W f(h_{t-1}) + W^{(h,x)} x_{[t]}$
- More chain rule, remember:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

- Each partial is a Jacobian:

$$\frac{d\mathbf{f}}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Una breve historia de la IA

Las técnicas mejoran, podemos aprender de los datos

Un caso práctico - Clasificación de imágenes



pre 2012

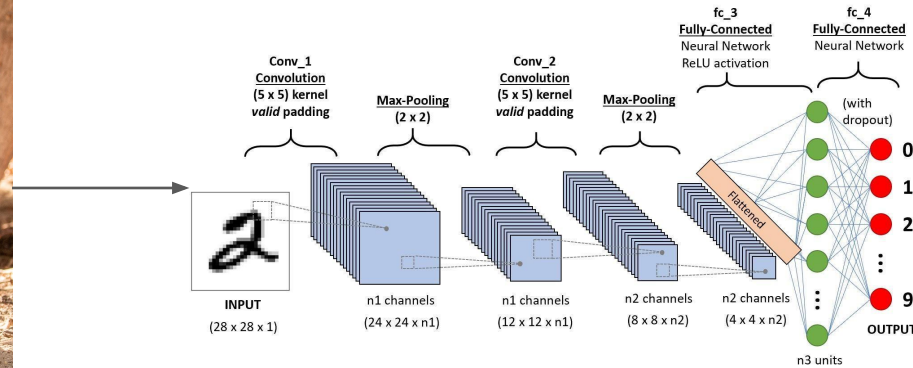
Una breve historia de la IA

Las técnicas mejoran, podemos aprender de los datos

Un caso práctico - Clasificación de imágenes



pre 2012



2012 - Alexnet

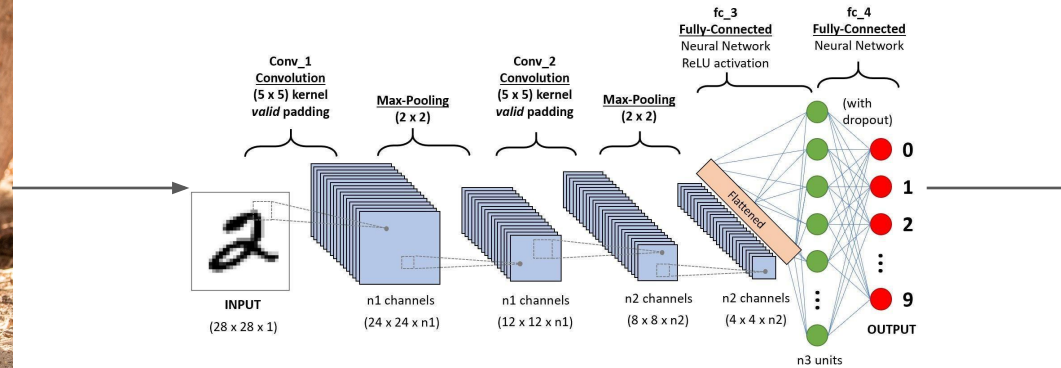
Una breve historia de la IA

Las técnicas mejoran, podemos aprender de los datos

Un caso práctico - Clasificación de imágenes



pre 2012



2012 - Alexnet



post 2012

Perceptrón



Hebbian Learning



Neuroplasticity

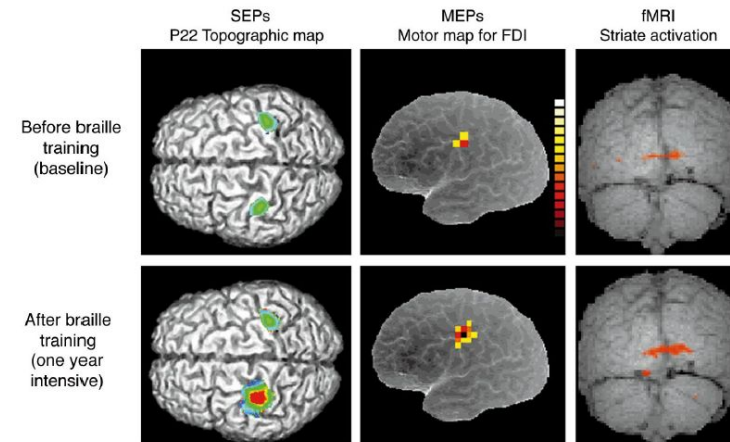
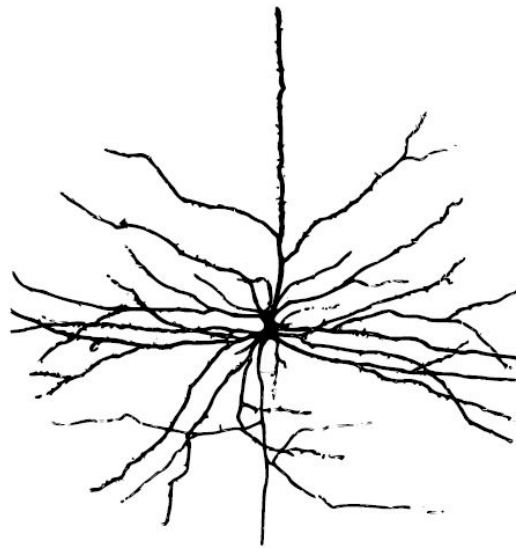
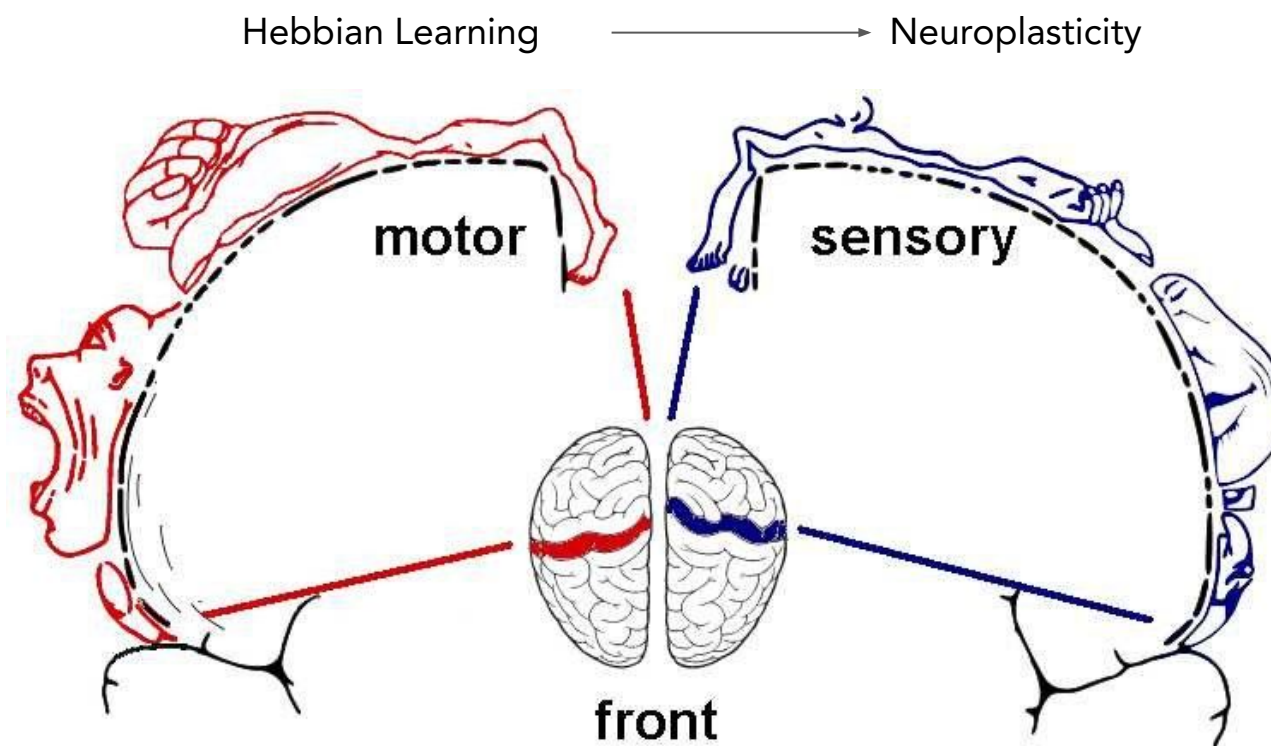
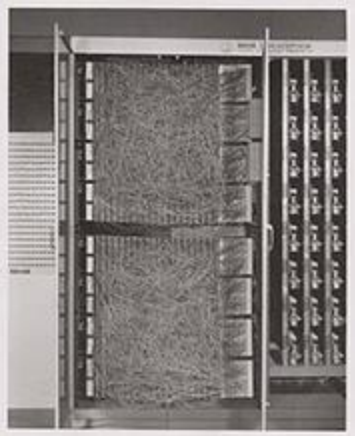


Fig. 1 Evidence of plasticity in sensory, motor and occipital cortex in early-blind subjects after learning Braille. Representative examples are shown from the studies performed before (top) and at the end of one year of learning Braille (bottom). The different studies were conducted on different subjects using somatosensory evoked potentials (SEPs) to mechanical stimuli to the index finger pad, motor mapping with TMS of the potentials evoked in the first dorsal interosseus (FDI) muscle (the side-to-side mover of the index finger), and functional magnetic resonance imaging (fMRI) while reading Braille characters. (Adapted from Refs 13,18.)



- Presentación del modelo **Perceptrón**.

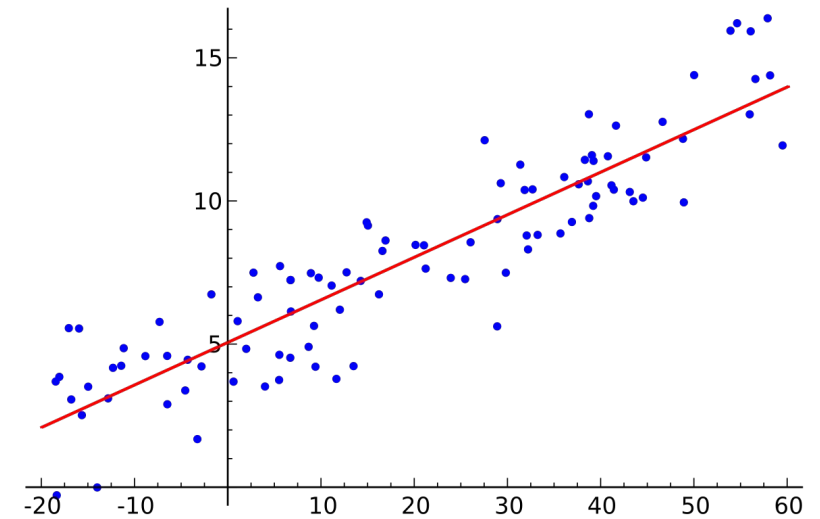
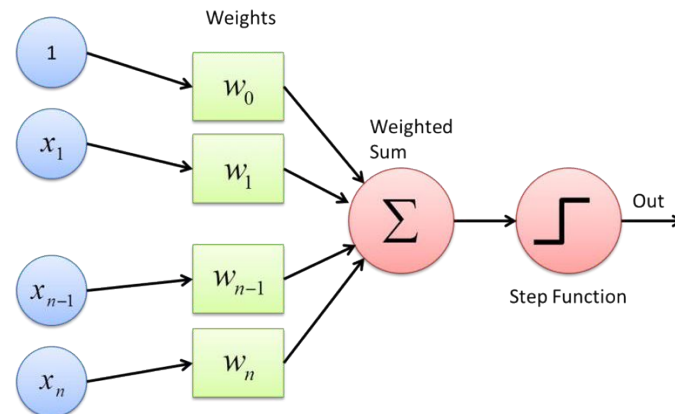


Rosenblatt (1958)

"the embryo of an electronic computer that [the Navy] expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence."

The Perceptron

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0, \\ 0 & \text{otherwise} \end{cases}$$



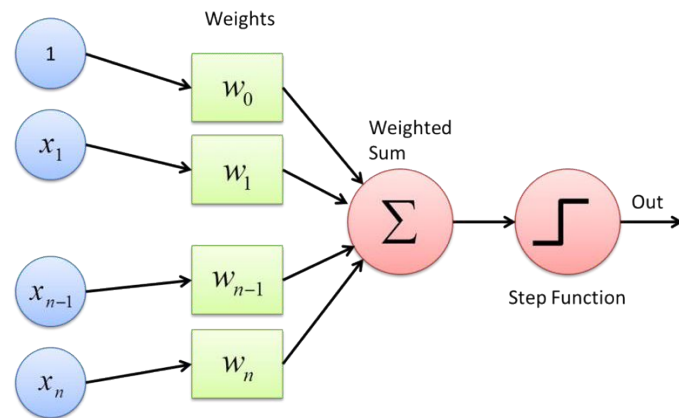
$$y = wx + b$$

$$y = \sum w_i x_i + b$$

Redes Neuronales - Intro a Deep Learning

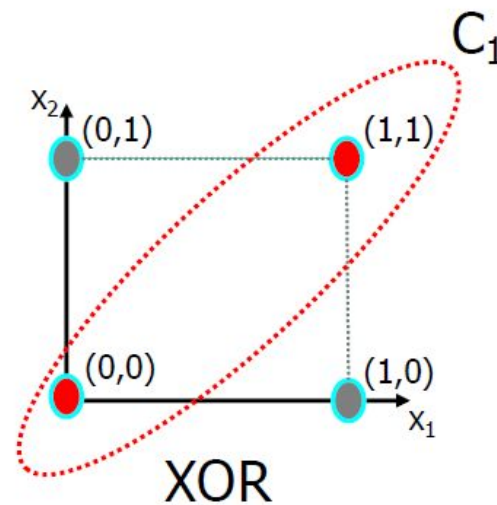


El problema XOR

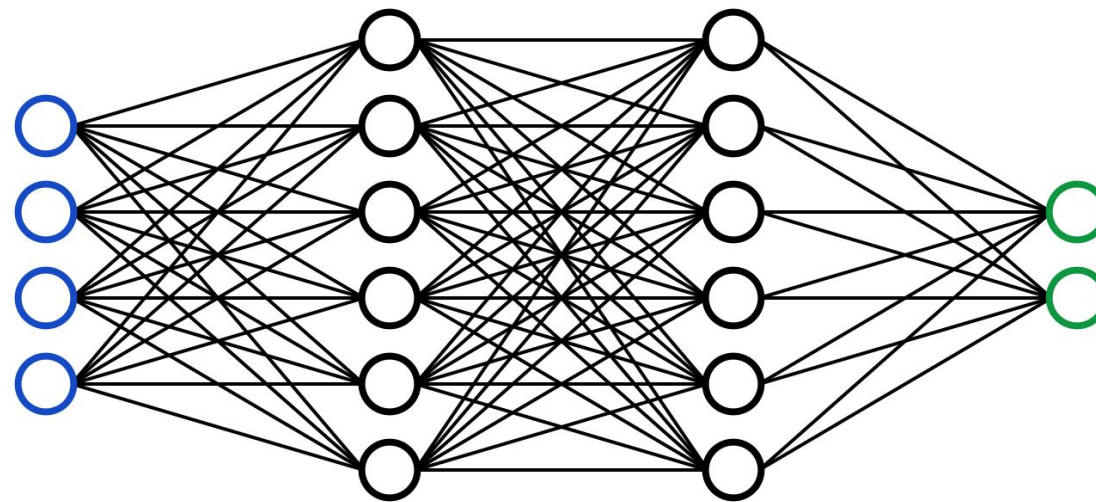
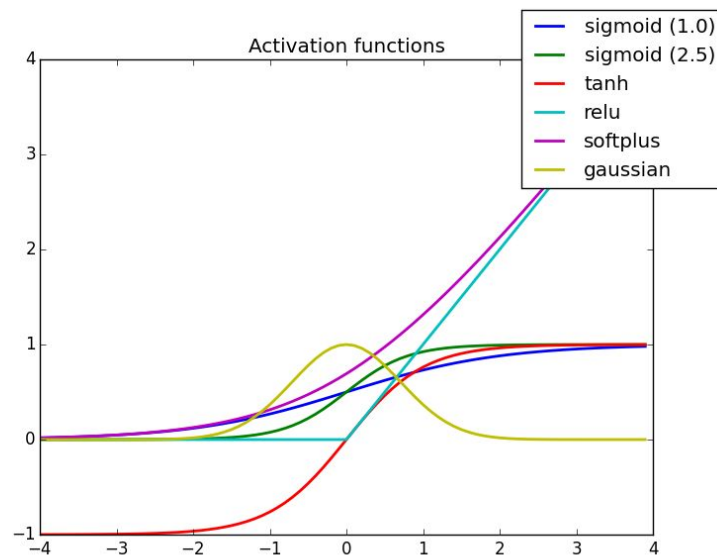


$$y = wx + b$$

$$y = \sum w_i x_i + b$$



La solución? -> **Deep Learning**



- Presentación de la técnica **backpropagation** para entrenamiento de redes neuronales con profundidad.

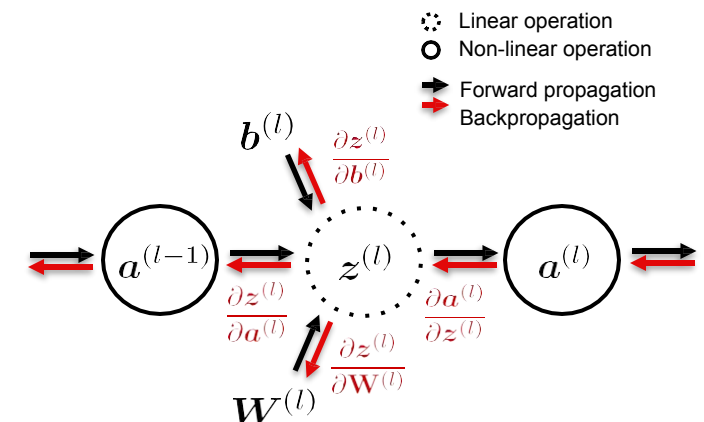
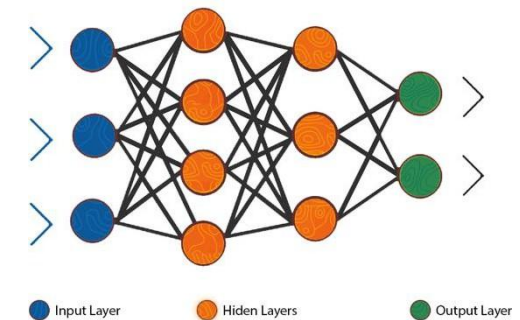
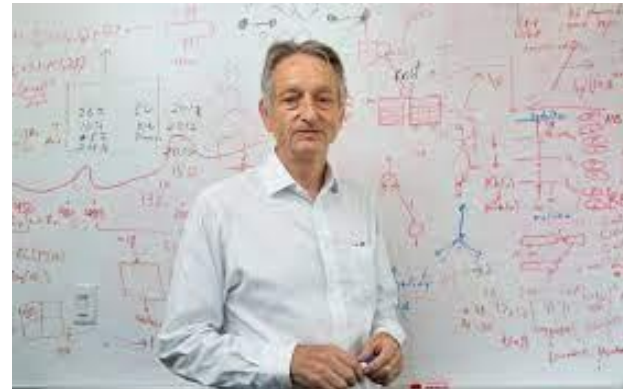
1986

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.



- Presentación de la arquitectura **LSTM** para procesamiento de datos secuenciales.

1997

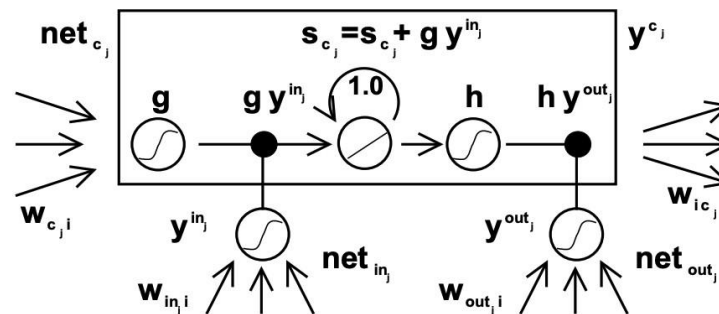
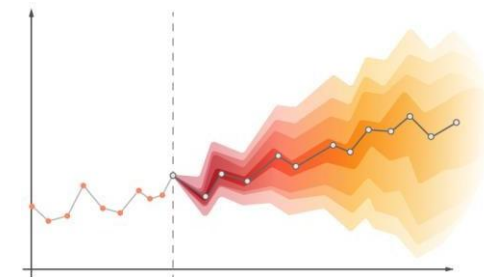
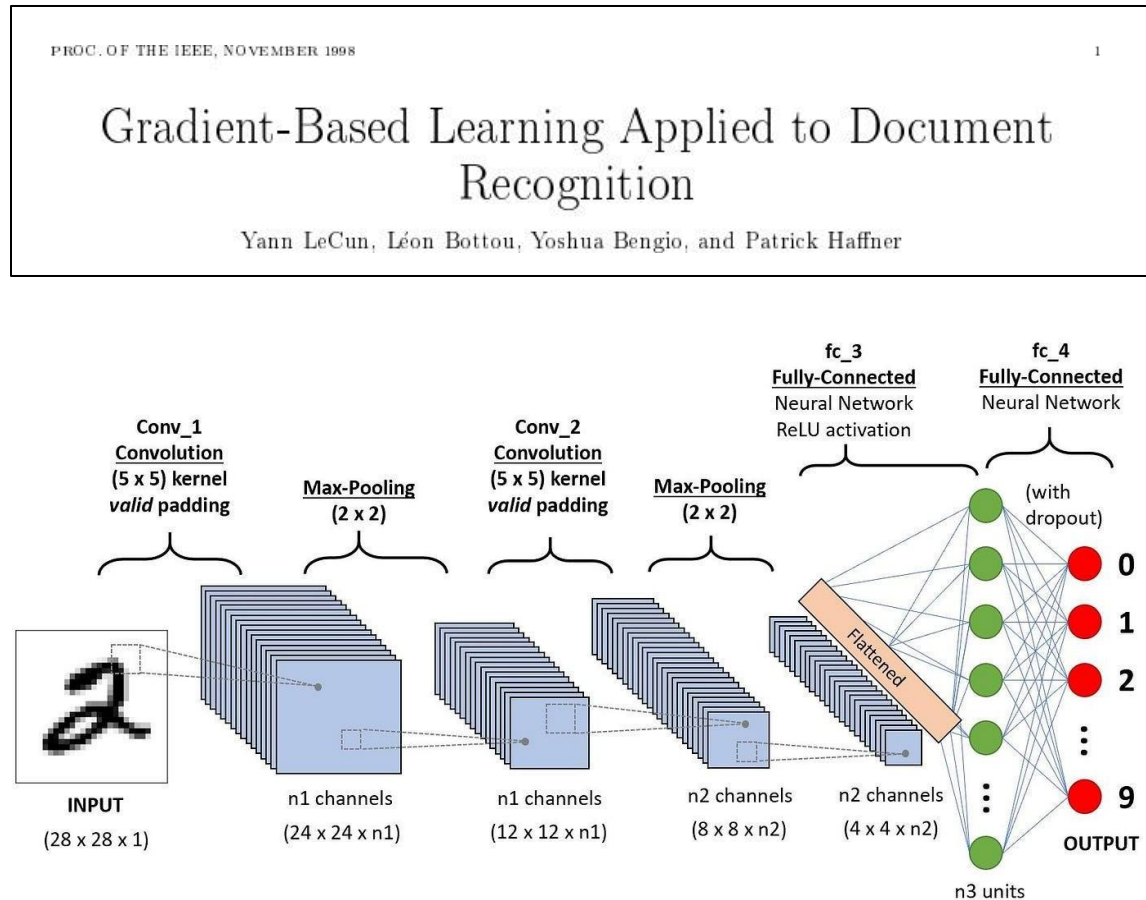


Figure 1: Architecture of memory cell c_j (the box) and its gate units in_j, out_j . The self-recurrent connection (with weight 1.0) indicates feedback with a delay of 1 time step. It builds the basis of the “constant error carousel” CEC. The gate units open and close access to CEC. See text and appendix A.1 for details.



- Presentación de las **Redes Neuronales Convolucionales (CNN)**.

1998



Yann LeCun



Yoshua Bengio



- Uso de **tarjetas gráficas** (GPU) para paralelización en el entrenamiento de Redes Neuronales.
- Permiten acelerar de manera drástica el entrenamiento en cantidades ingentes de datos.

2012

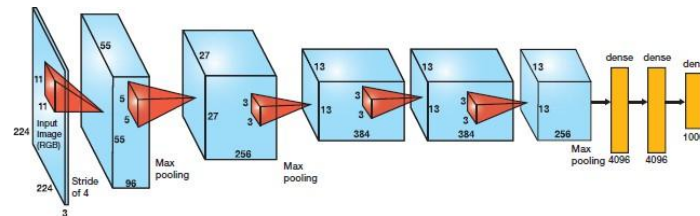


ImageNet Classification with Deep Convolutional Neural Networks

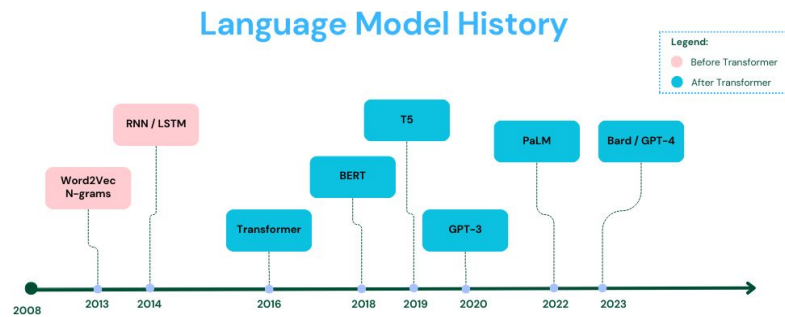
Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca



- Presentación de la arquitectura **Transformer** para procesamiento de datos secuenciales.



2016

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

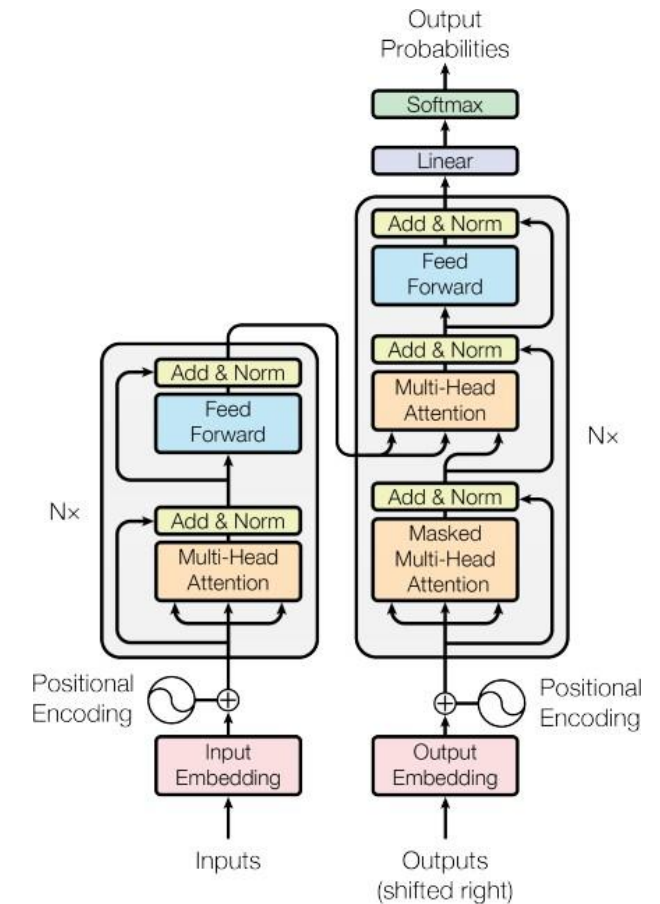
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

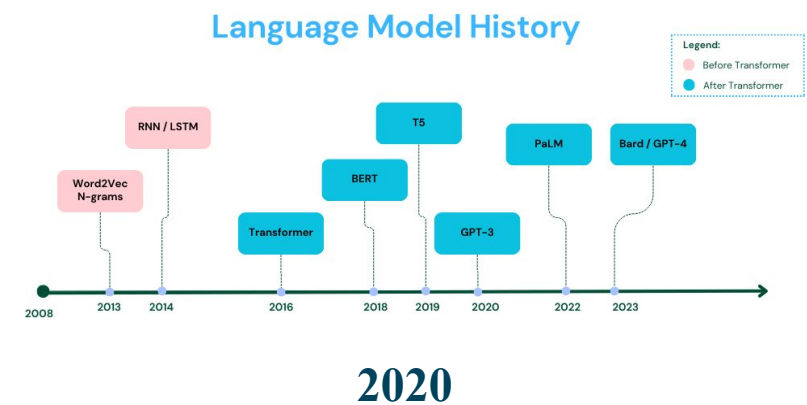
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

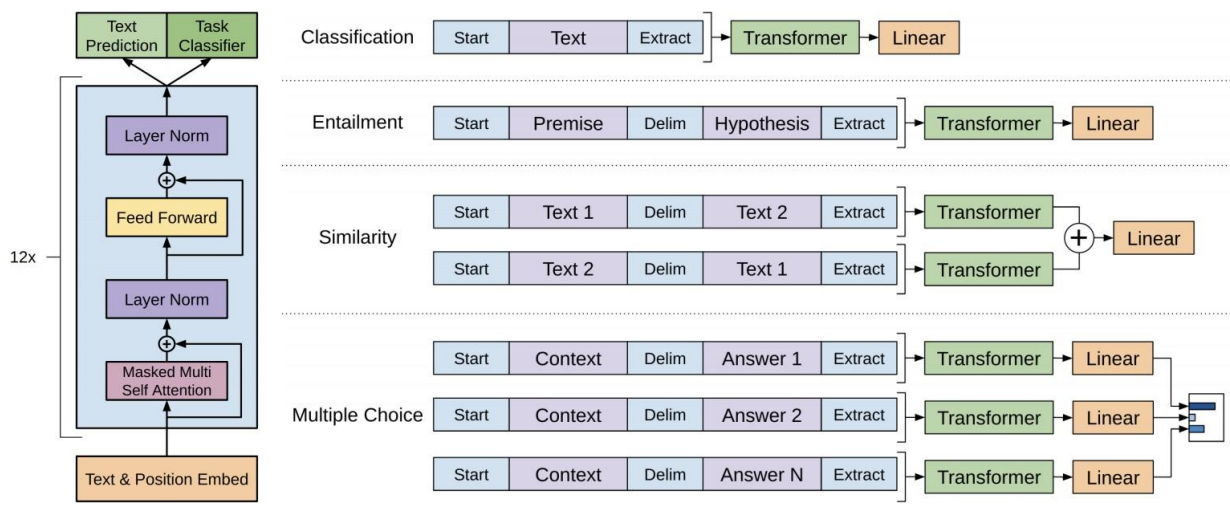
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

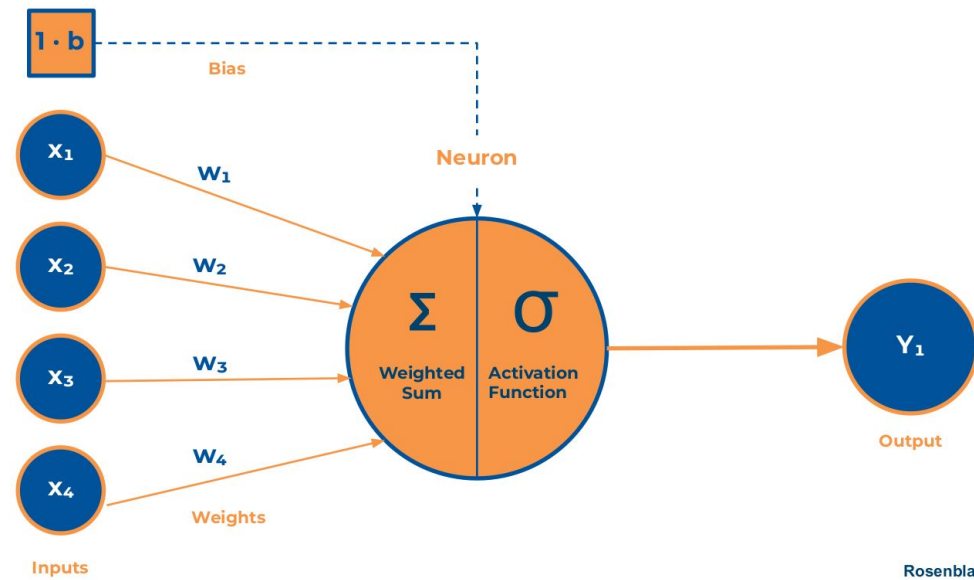


- Presentación del modelo **GPT** para generación de texto conversacional.

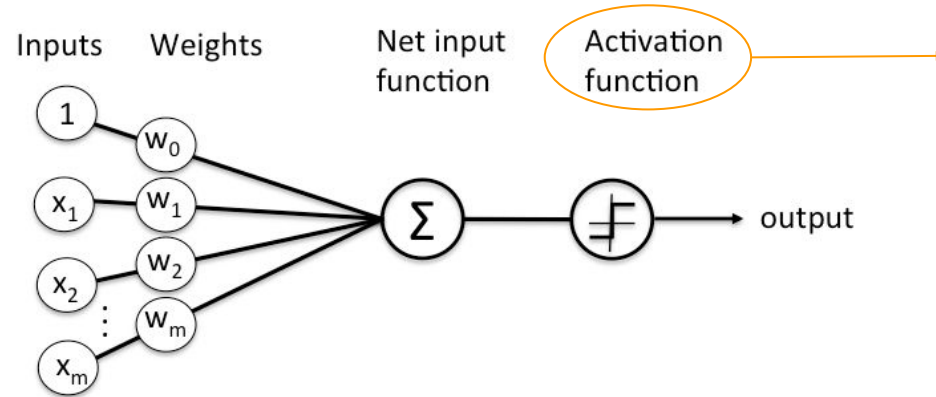


Language Models are Few-Shot Learners				
Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan†	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	
OpenAI				

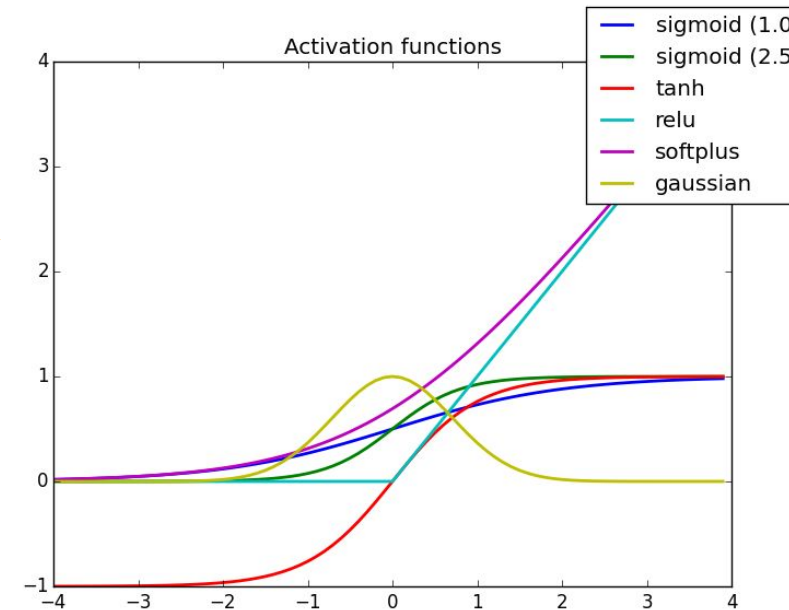




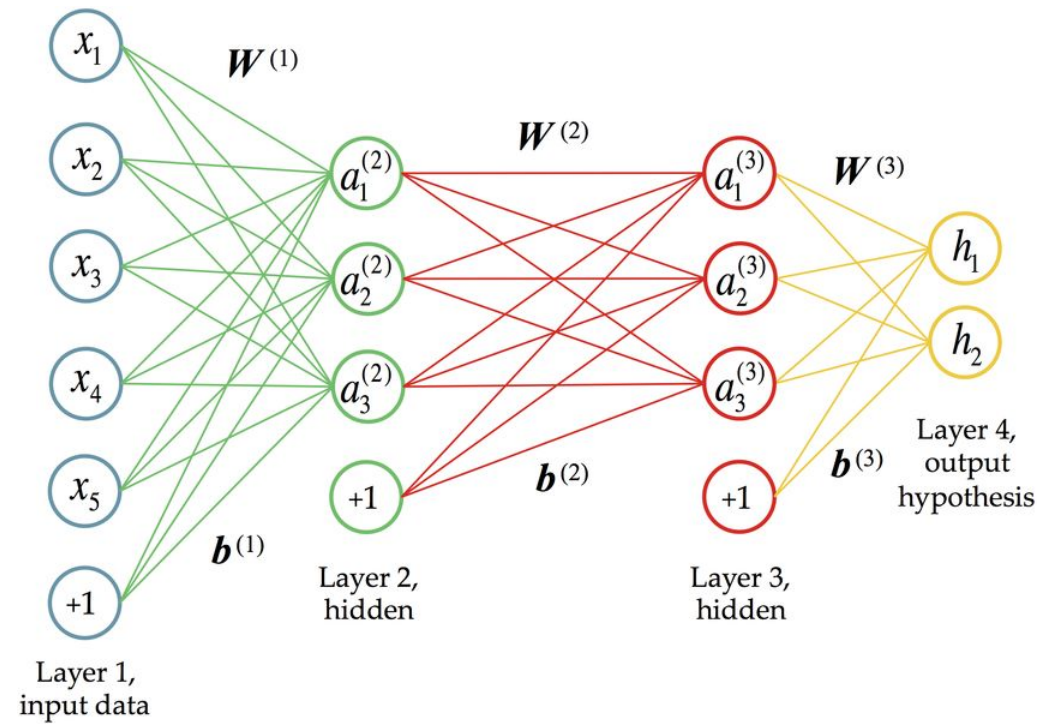
Rosenblatt, F. (1957)

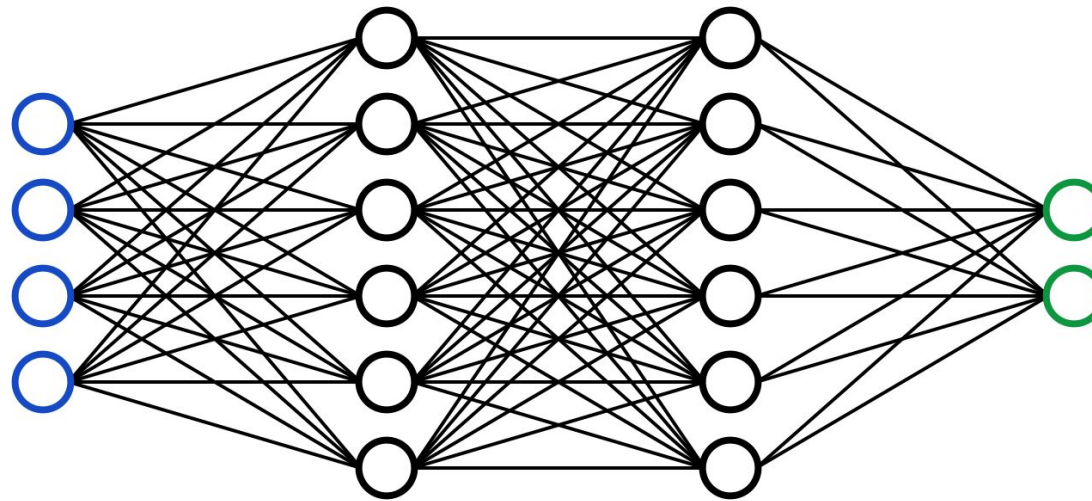


$$y = \sum w_i x_i + b$$



$$y = \sum f(w_i x_i + b)$$





Lots of data & train time
Subject to overfitting
Universal problem solvers
Importance of weights - backprop

Deep Learning

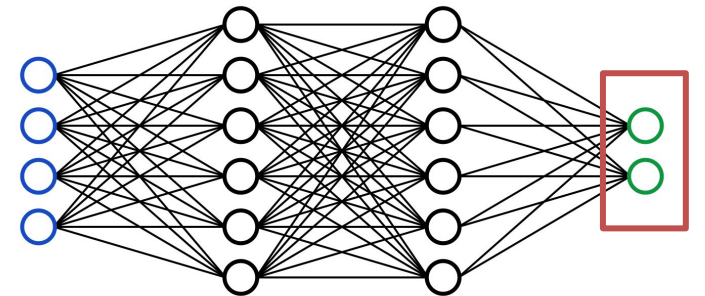


Función de pérdida: compara la diferencia entre el resultado calculado y el objetivo. Por ejemplo: SSE

$$\mathcal{L}^{(p)}(W) = \sum_{i=1}^{yx} \left(t_i^{(p)} - y_i^{(p)} \right)^2$$

Función de coste: calcula el error global utilizando todo el conjunto de datos. Por ejemplo: MSE.

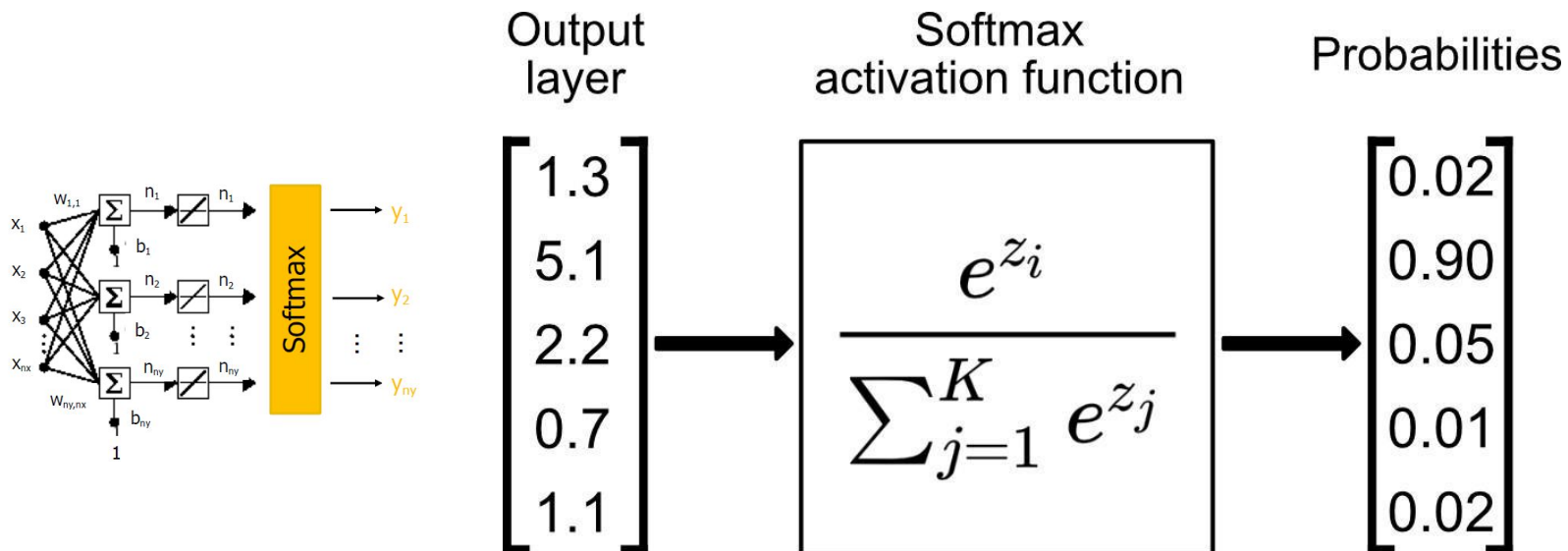
$$J(W) = \text{MSE}(W) = \frac{1}{m} \sum_{p=1}^m E^{(p)}(W)$$



id	color
1	red
2	blue
3	green
4	blue

One Hot Encoding

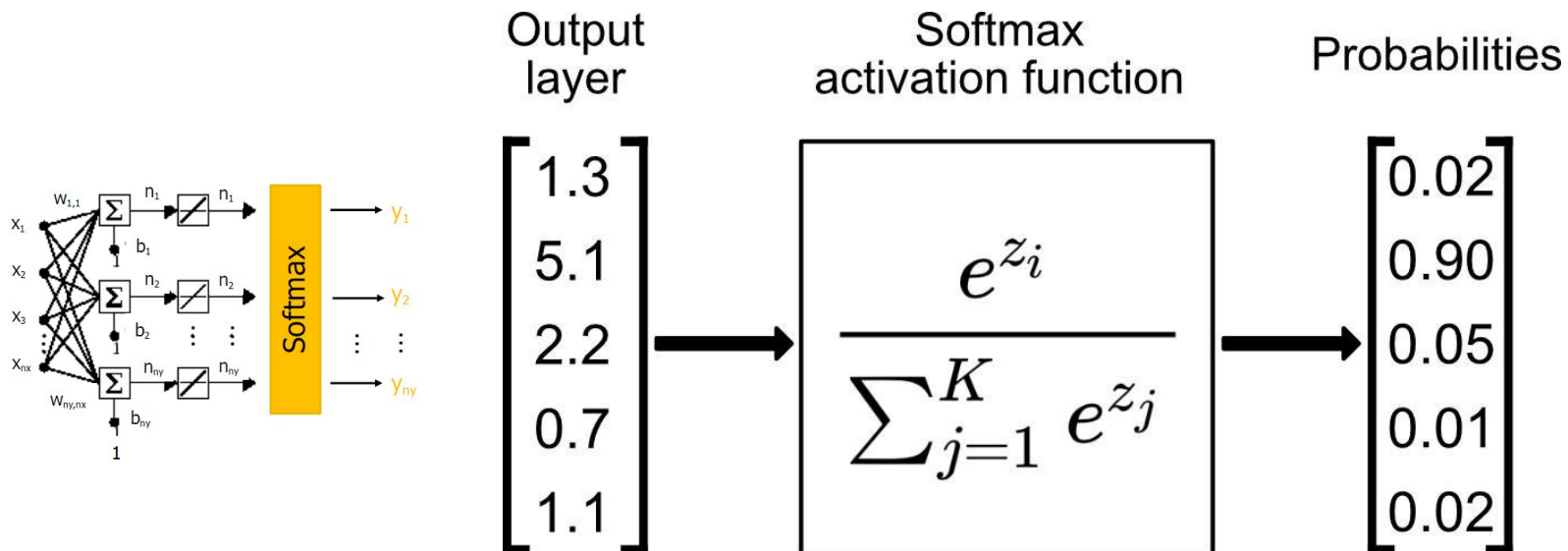
id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0



- Se utiliza en la última capa de la red para la predicción categórica.
- Convierte valores numéricos en probabilidades.
- Se utiliza junto con one-hot encoding.

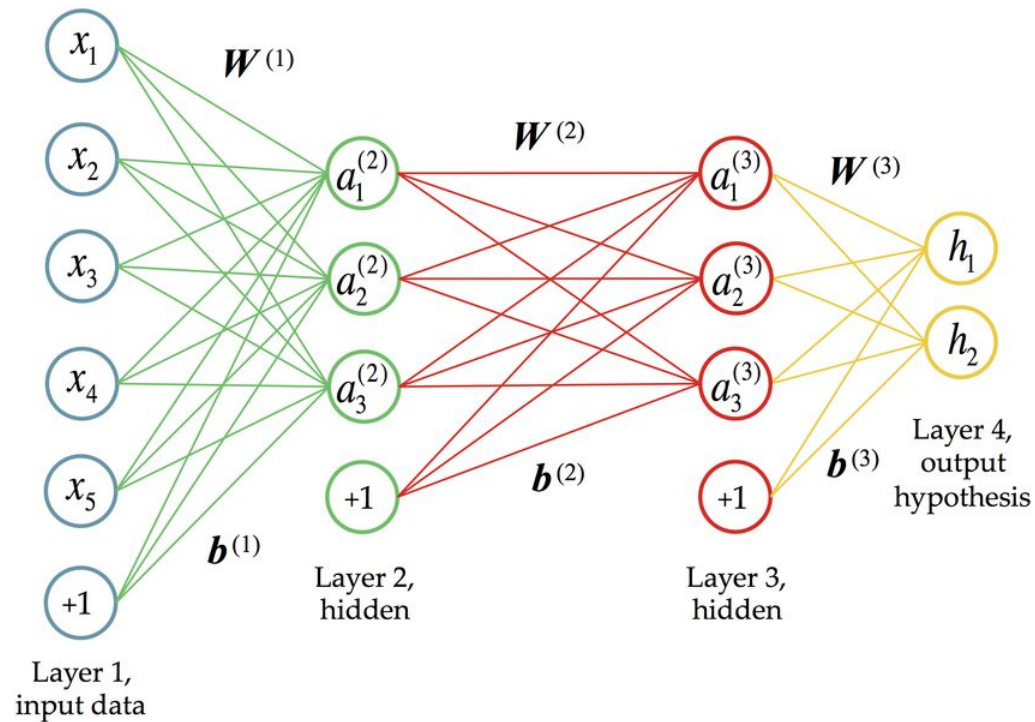
Concepto de:

- Confianza en la predicción
- Top K
-



- Se utiliza en la última capa de la red para la predicción categórica.
- Convierte valores numéricos en probabilidades.
- Se utiliza junto con one-hot encoding.

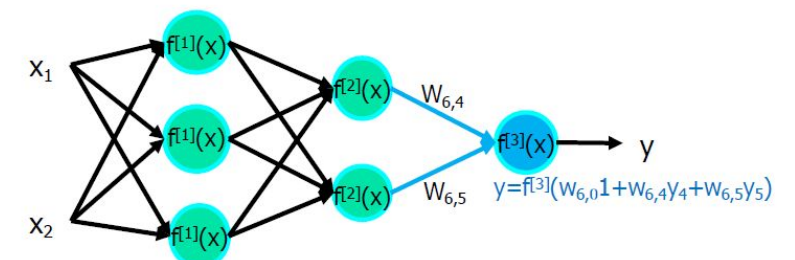
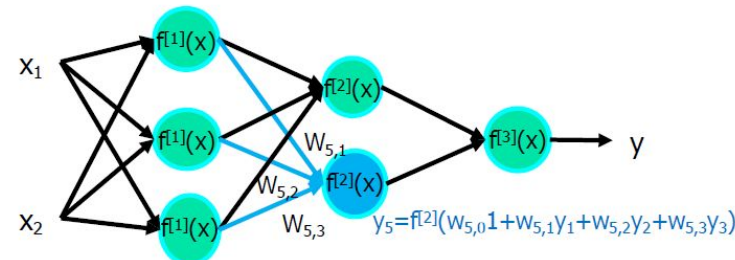
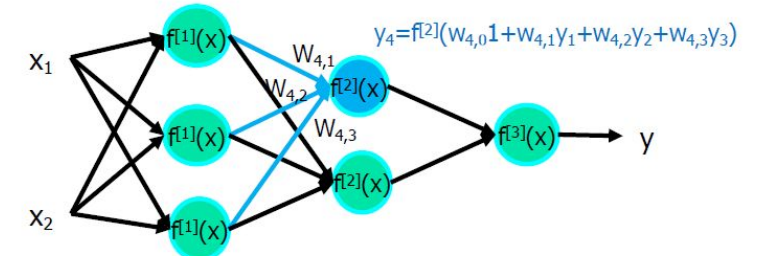
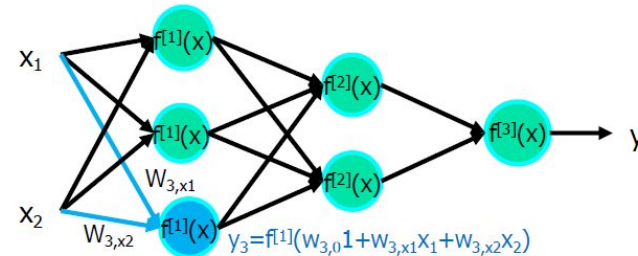
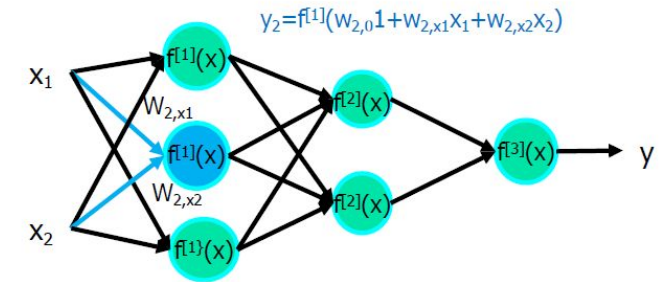
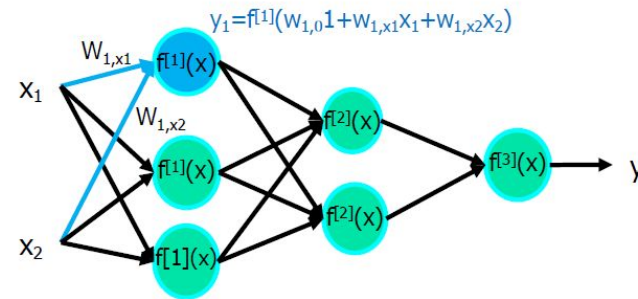
Entrenamiento de redes de neuronas -> aprender los pesos -> backpropagation



$$\mathcal{L}^{(p)}(W) = \sum_{i=1}^{y \times x} \left(t_i^{(p)} - y_i^{(p)} \right)^2$$

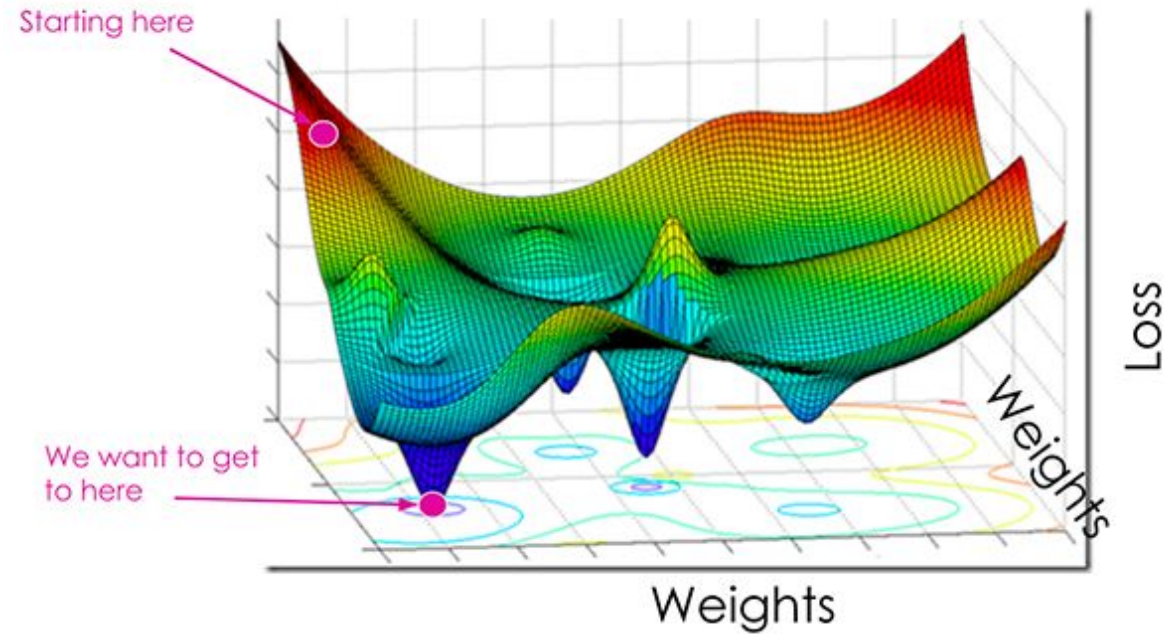
$$J(W) = \text{MSE}(W) = \frac{1}{m} \sum_{p=1}^m E^{(p)}(W)$$

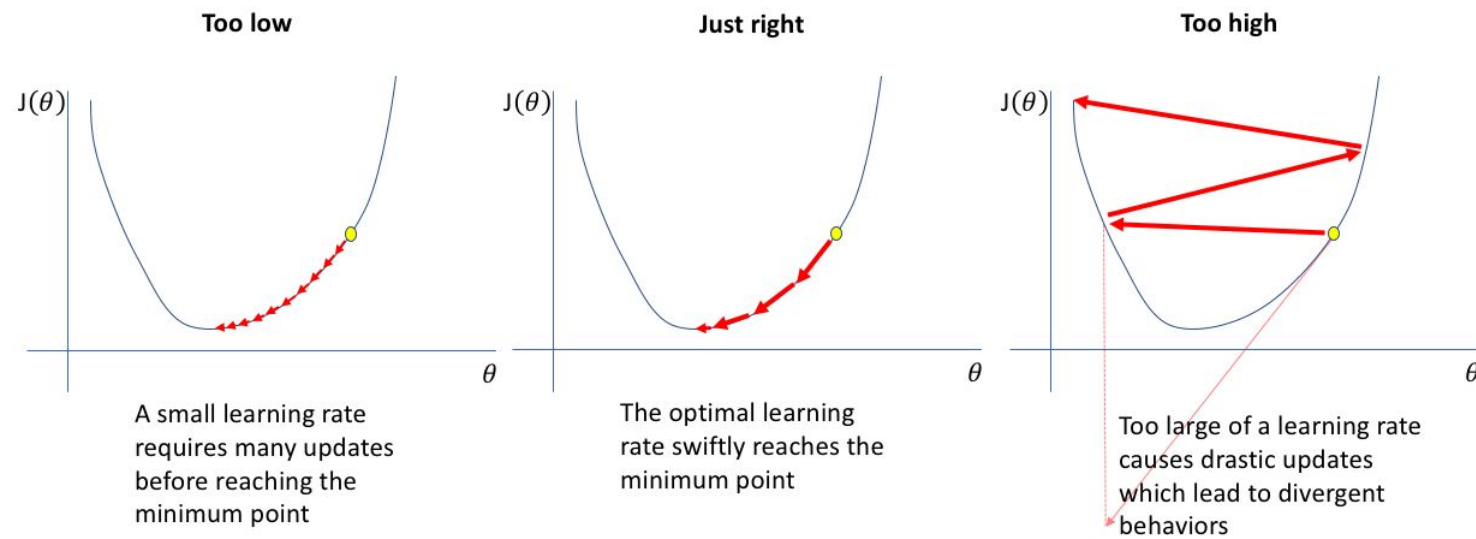
1. Inicializar los parámetros con números aleatorios
2. Calcular la predicción del conjunto de datos de entrenamiento



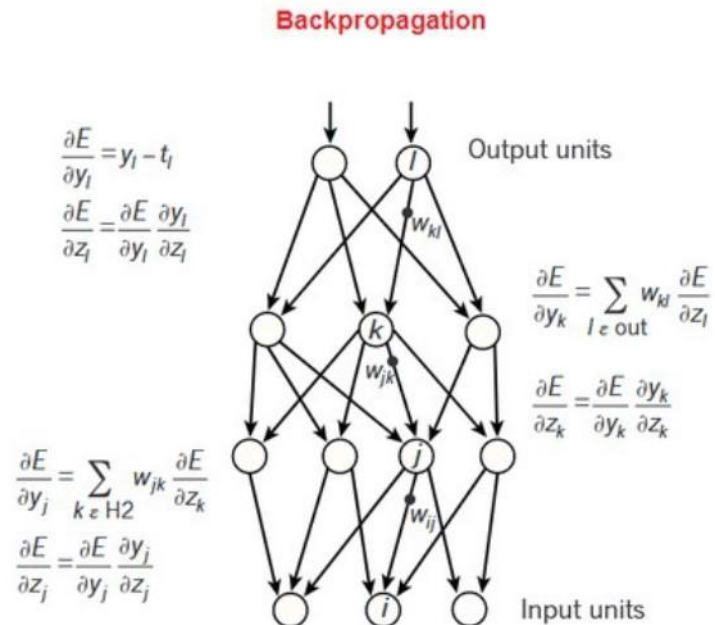
3. Calcular la función de coste

4. Encontrar el gradiente de la función de coste



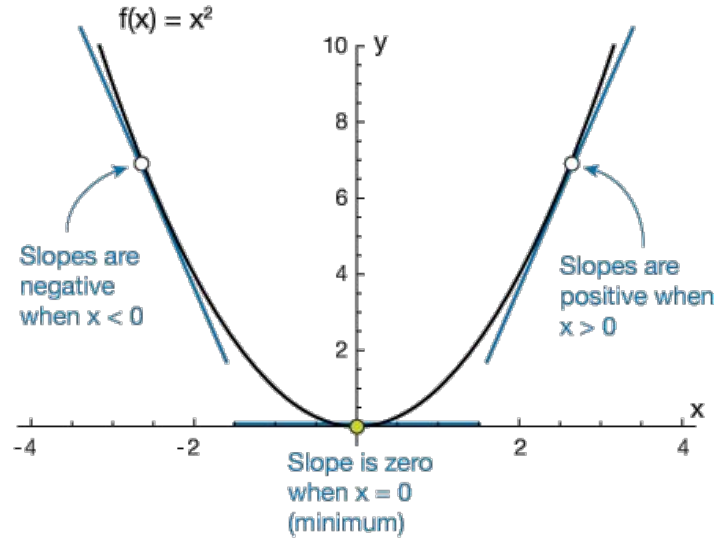


5. Cambiar los pesos en función de una cantidad λ según la dirección del gradiente



6. Propagar el error hacia las capas anteriores de la red:
Backpropagation.

Importancia de las derivadas y la regla de la cadena



If we have a function $A[B]$ of a function $B[C]$

$$A[C] = A[B[C]]$$

then the derivative of A with respect to C is

$$A'[C] = A'[B] \times B'[C]$$

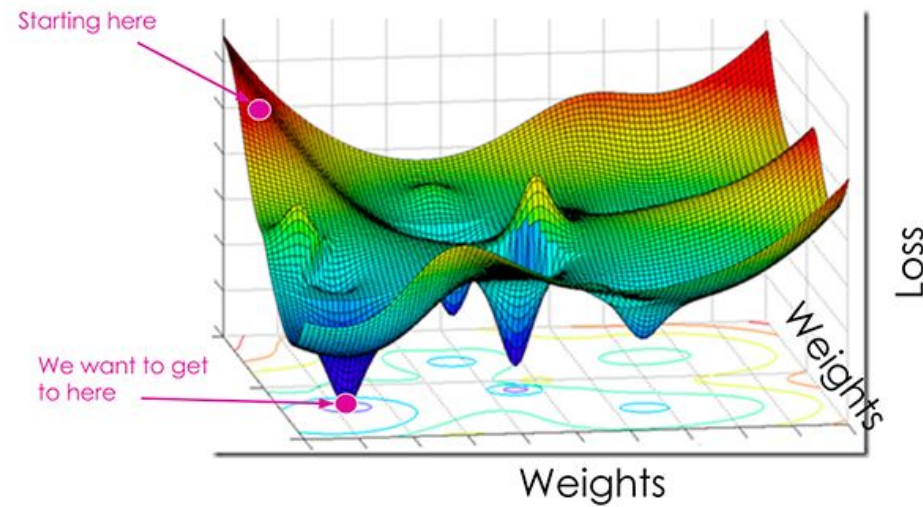
If we have a function $A[B[C[D[E[F]]]]]$

We can keep applying the chain rule to obtain the derivative:

$$A'[F] = A'[B] \times B'[C] \times C'[D] \times D'[E] \times E'[F]$$

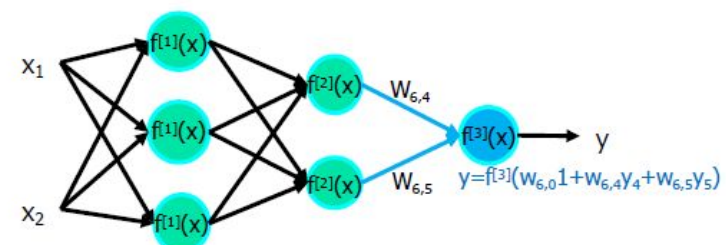
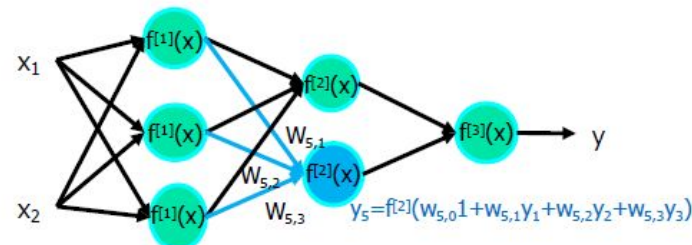
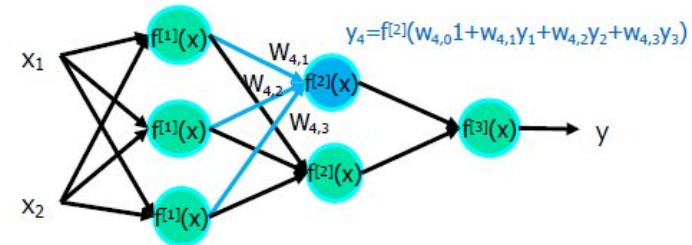
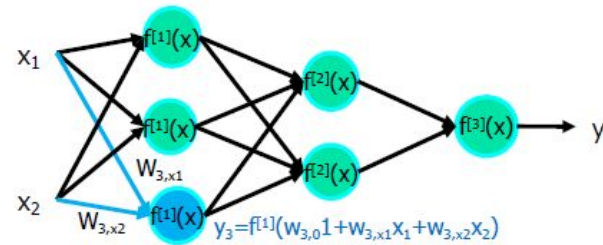
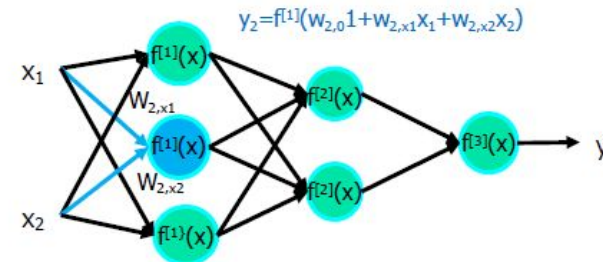
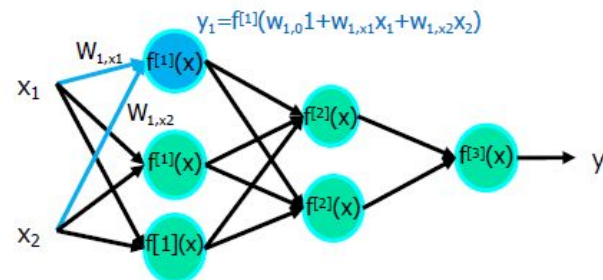
Entendiendo derivadas parciales

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \dots \right)$$

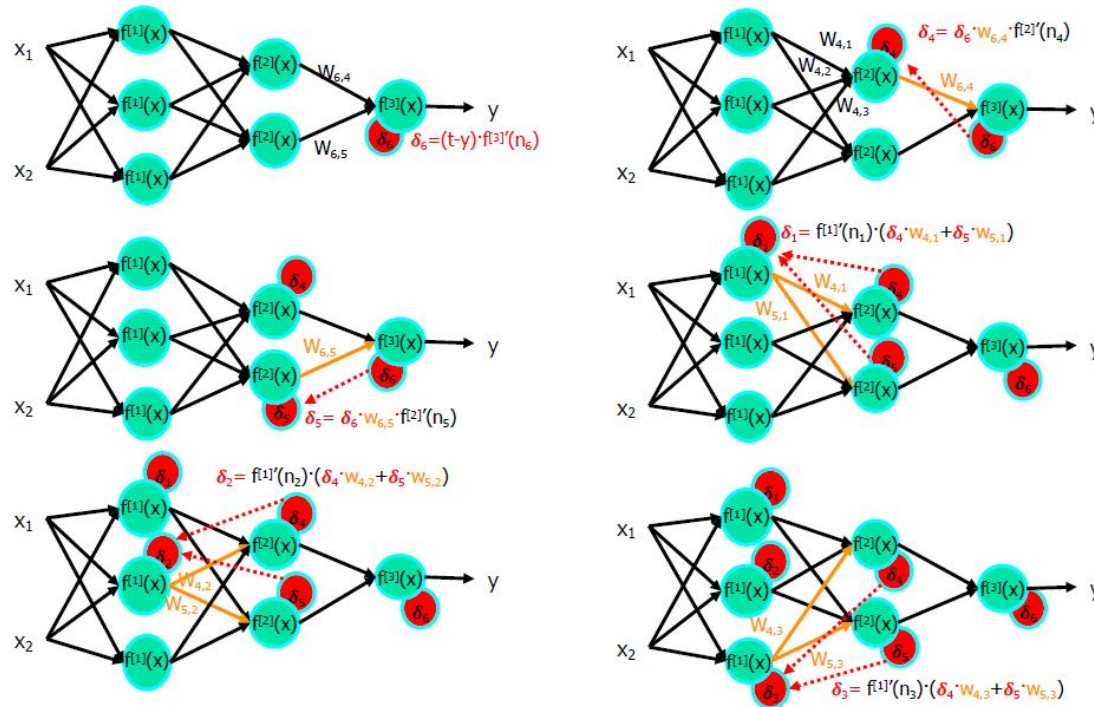


Assuming squared-error loss and single-example updates

primer paso - feed forward



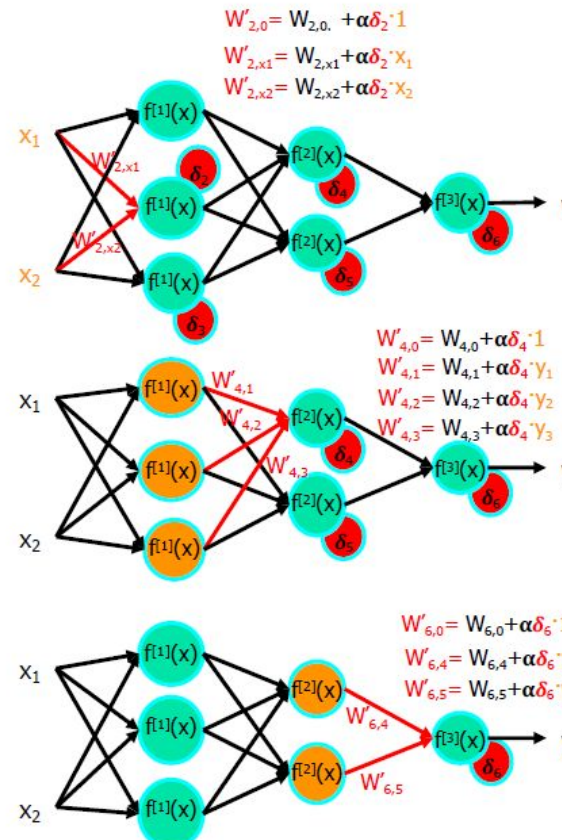
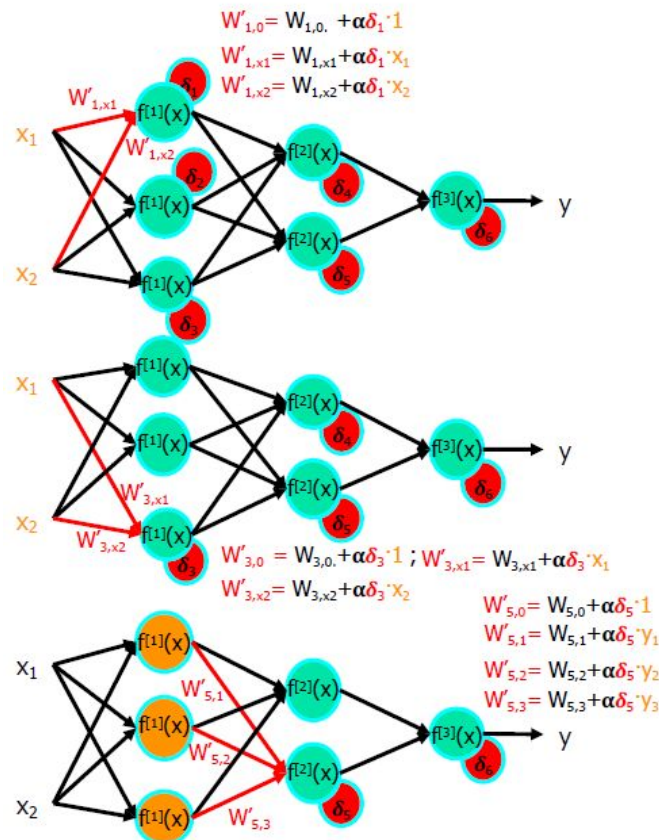
Calculamos los errores por cada neurona mediante backprop



$$\delta_i^{(p)} = (t_i^{(p)} - y_i^{(p)}) \cdot f'(\text{net}_i^{(p)}).$$

$$\delta_i^{(p)} = f'(\text{net}_i^{(p)}) \sum_k \delta_k^{(p)} \cdot W_{ki}.$$

Actualizamos los pesos



$$\Delta^{(p)} W_{ij} = \alpha \cdot \delta_i^{(p)} \cdot y_j^{(p)}$$

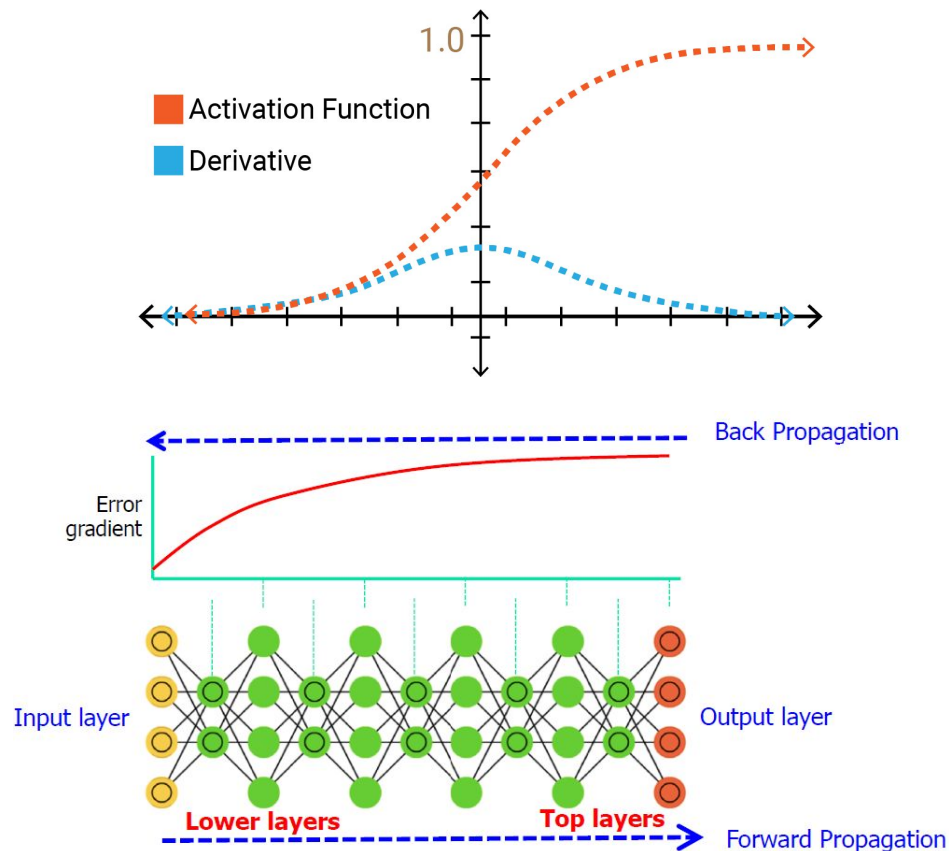
$$W'_{ij} = W_{ij} + \Delta^{(p)} W_{ij}$$

$$w_{ij} \leftarrow w_{ij} - \alpha \frac{\partial L}{\partial w_{ij}}$$

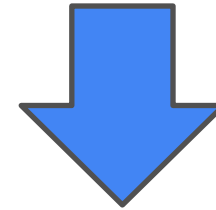
$$\frac{\partial L}{\partial w_{ij}} = \delta_i \cdot a_j$$

weight gradient =
destination neuron error
× previous neuron activation

Volvemos a las funciones de activación



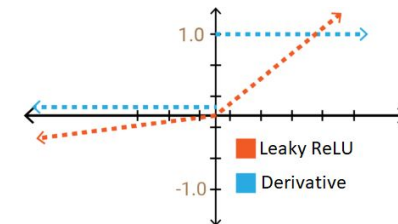
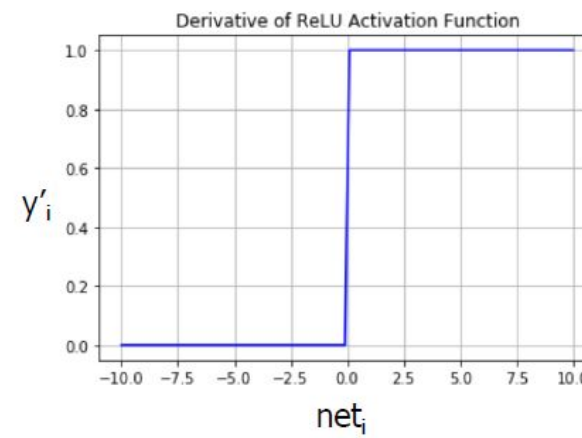
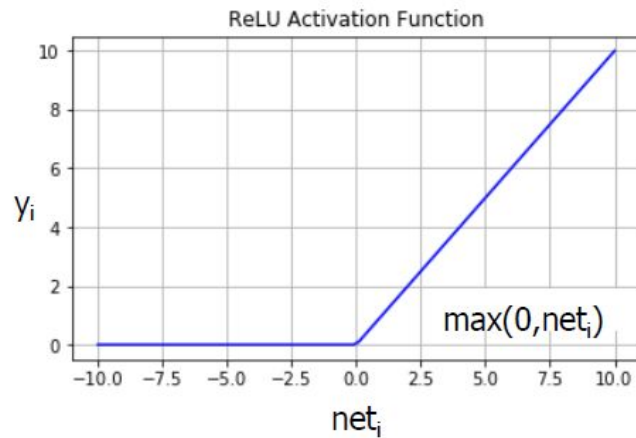
Las funciones de activación saturadas tienen gradientes que tienden a cero.



Las redes profundas tienen dificultades para aprender en sus capas iniciales.

ReLU

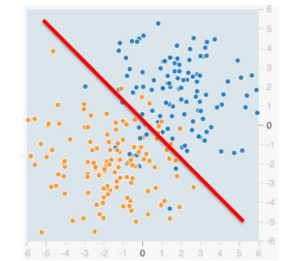
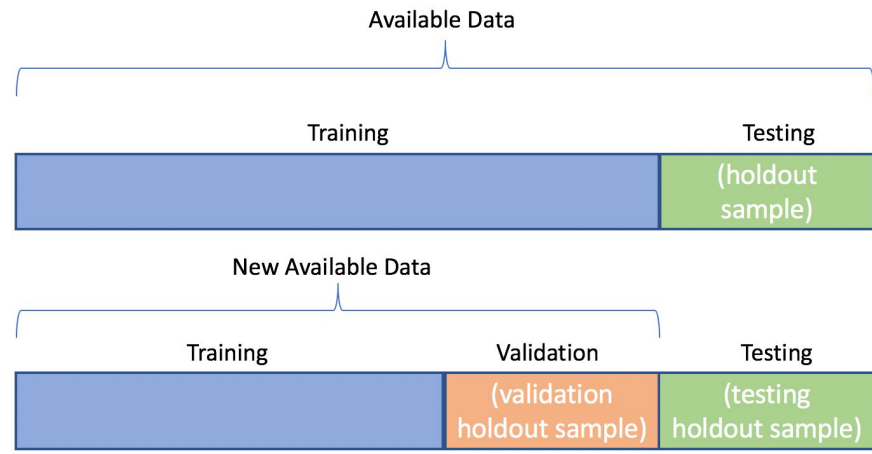
- No se satura.
- Es muy sencillo.
- Es rápido computacionalmente.



Entrenamiento y técnicas

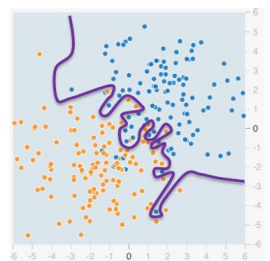


¿Como entrenamos una red de neuronas?



The model does not fit well the training set

High bias

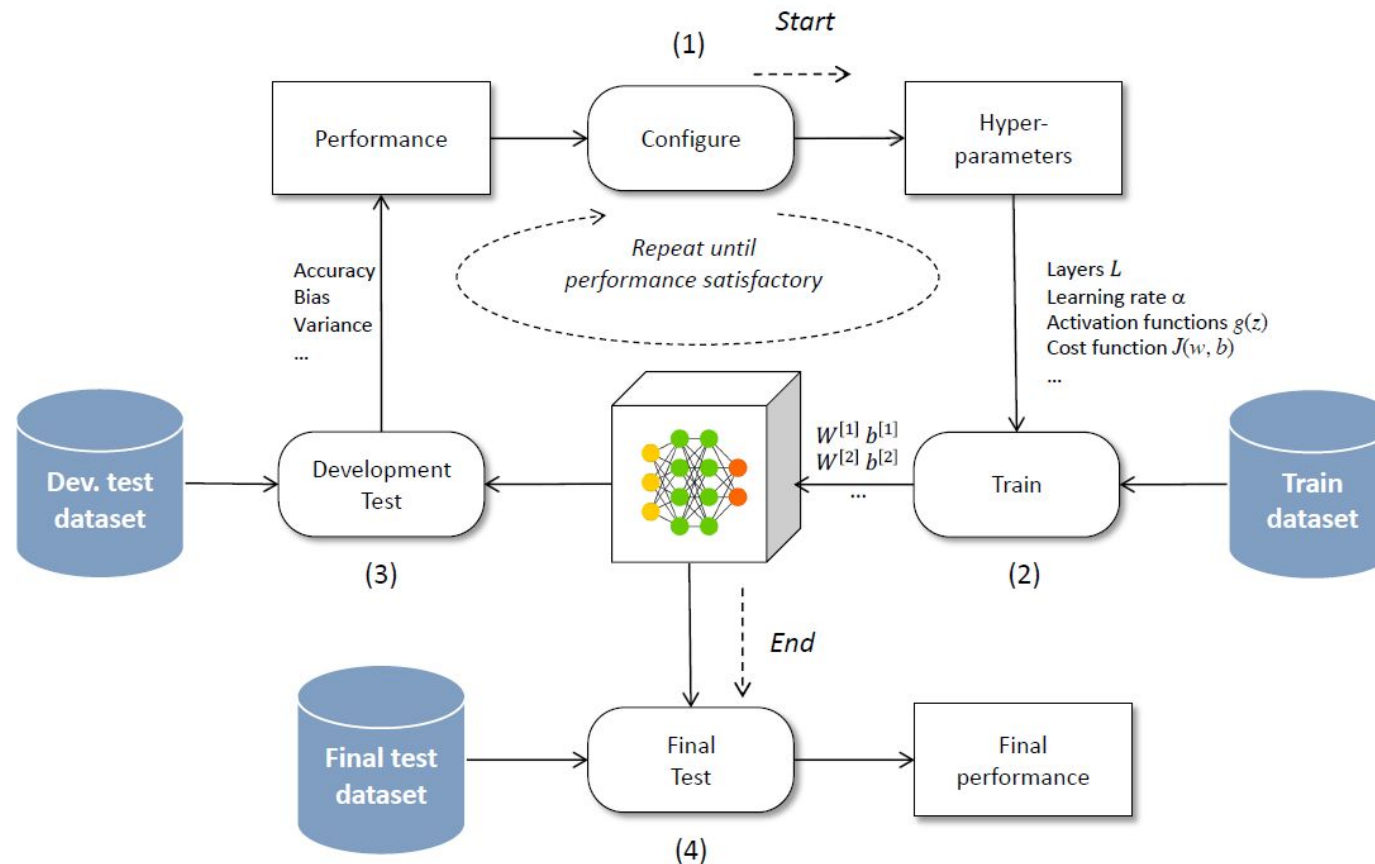


The model fits well the training set but the generalization can be poor

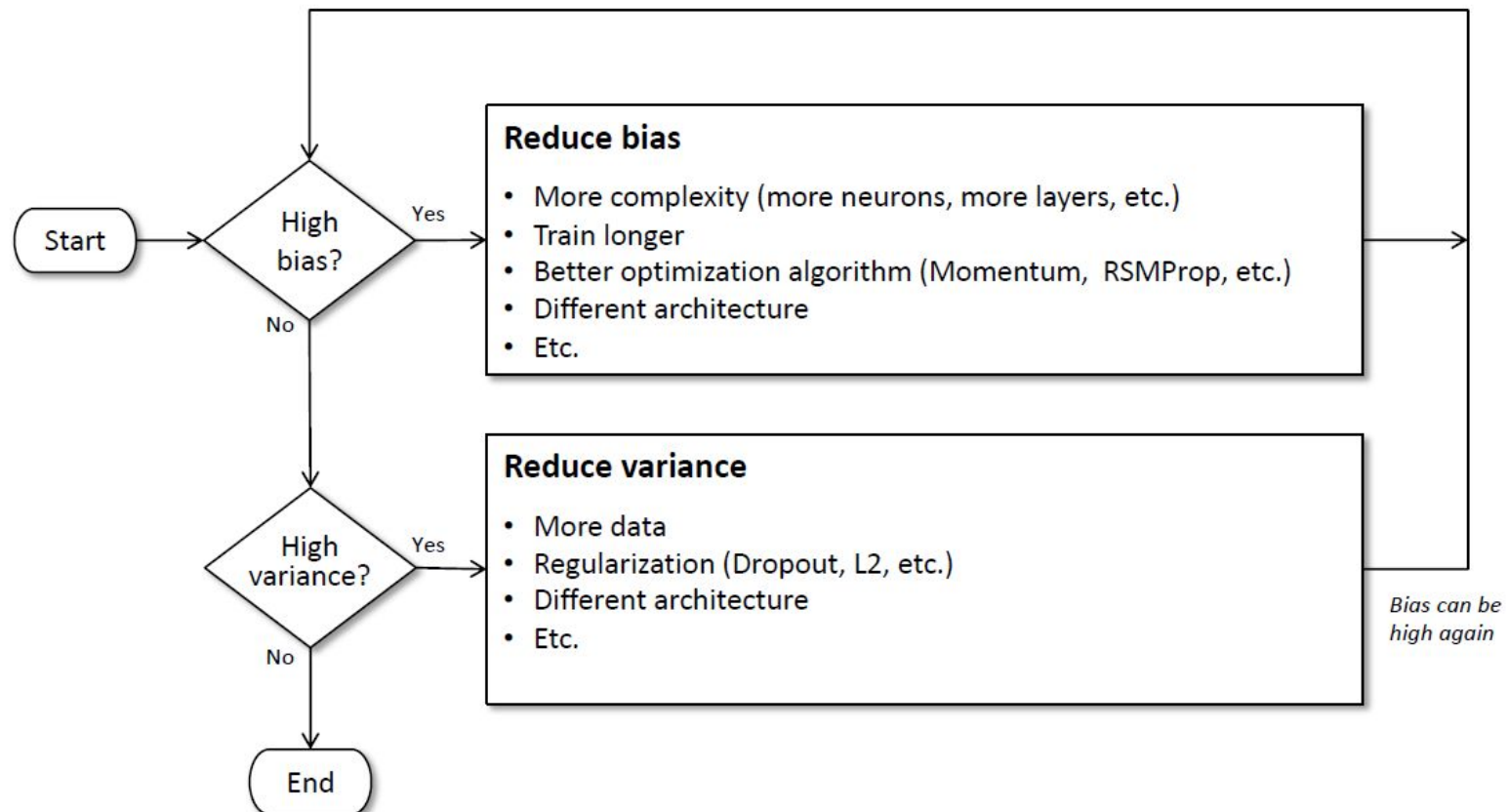
High variance

E_{train}	E_{test}	
1 %	15%	Low-bias, high-variance
15 %	16%	High-bias, low-variance
15 %	30%	High-bias, high-variance
0.5 %	1%	Low-bias, low-variance

Como saber donde parar de entrenar una red de neuronas?

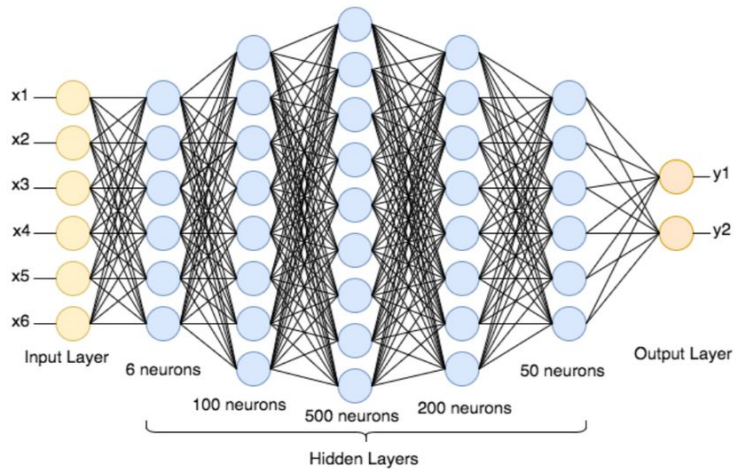


No hay una estrategia clara, pero podemos seguir ciertas reglas básicas



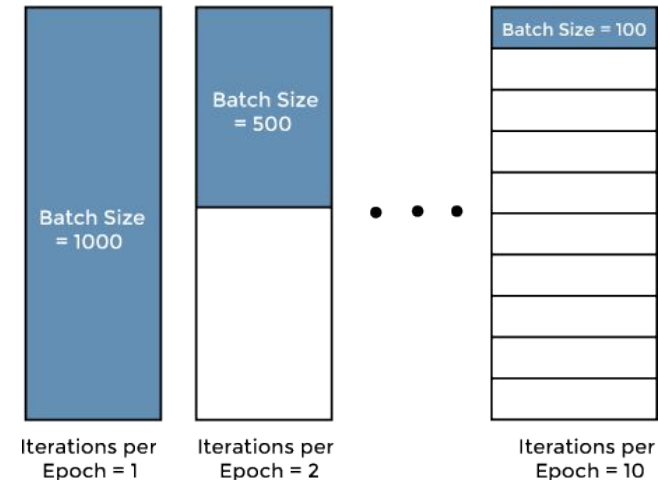
¿Qué es un batch?

Es un subconjunto del conjunto total de datos que se utiliza en cada iteración para calcular el gradiente y actualizar los pesos del modelo.



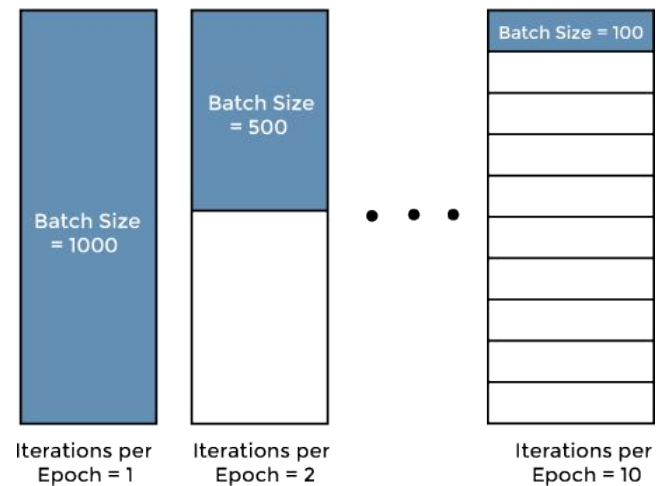
¿Por qué usamos batches?

- Permiten **entrenar modelos en paralelo** en aceleradores como **GPUs**.
- Reduce el coste computacional respecto a usar todo el dataset en cada paso.

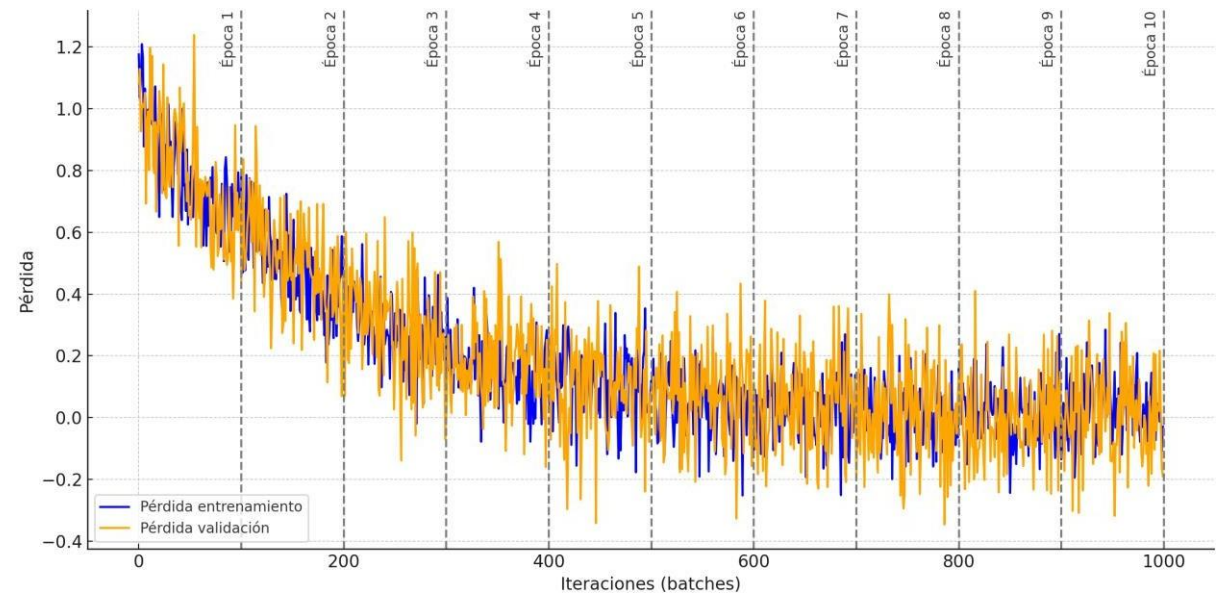


¿Qué es una época?

Se llama **época** a un **recorrido completo** del modelo por todo el conjunto de entrenamiento.

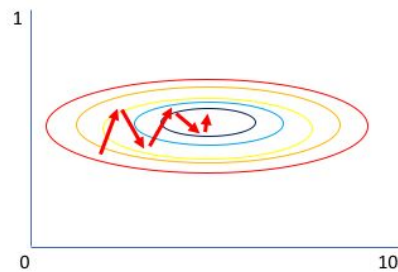


- Durante una época, el modelo **ve cada ejemplo de entrenamiento una vez**.
- Si entrenamos durante 10 épocas, el modelo habrá visto el dataset completo **10 veces**.
- Cada época se divide en **batches**, que se procesan uno a uno.

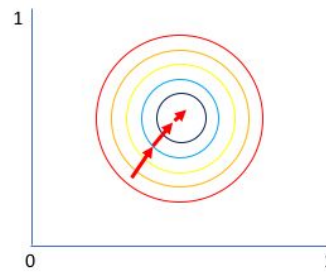


Que no se nos olvide normalizar

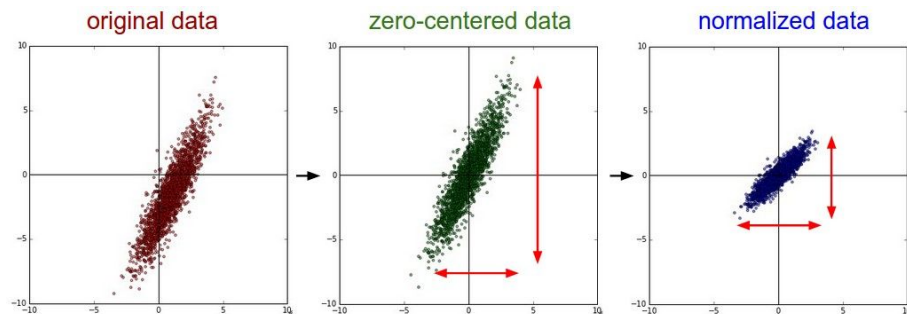
Why normalize?



Gradient of larger parameter dominates the update



Both parameters can be updated in equal proportions



- Las neuronas saturadas no pueden aprender.
- Las variables más grandes dominan el aprendizaje.
- Evita los sesgos en los datos

Medidas de regularización: L1 y L2

L1 Regularization

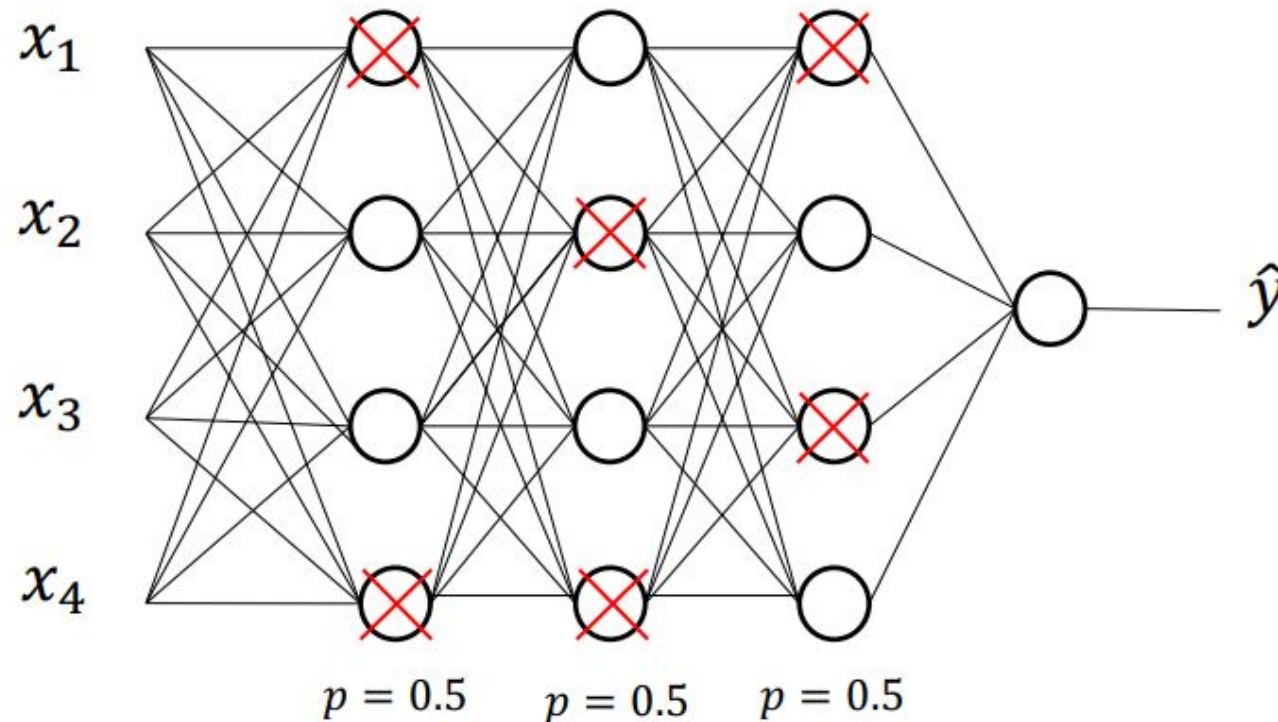
$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

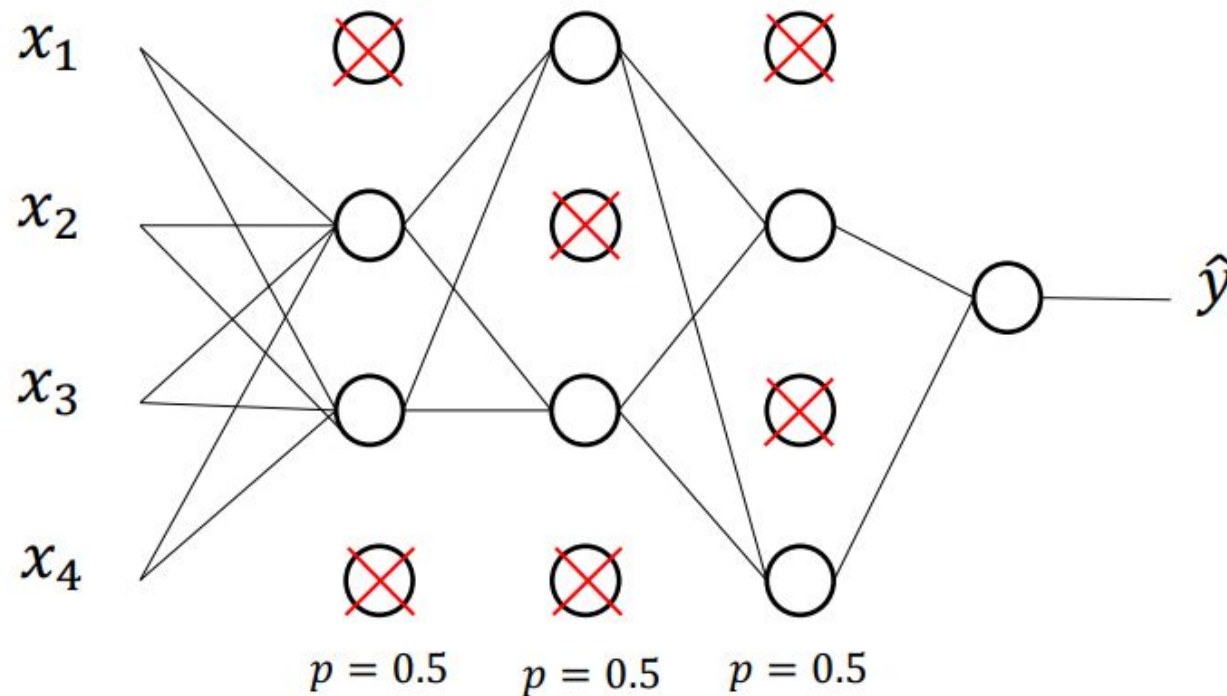
$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

- El sobreajuste se atribuye a veces a pesos elevados, los cuales se producen a lo largo del tiempo.
- Podemos evitarlo manteniendo los pesos bajos en la red.

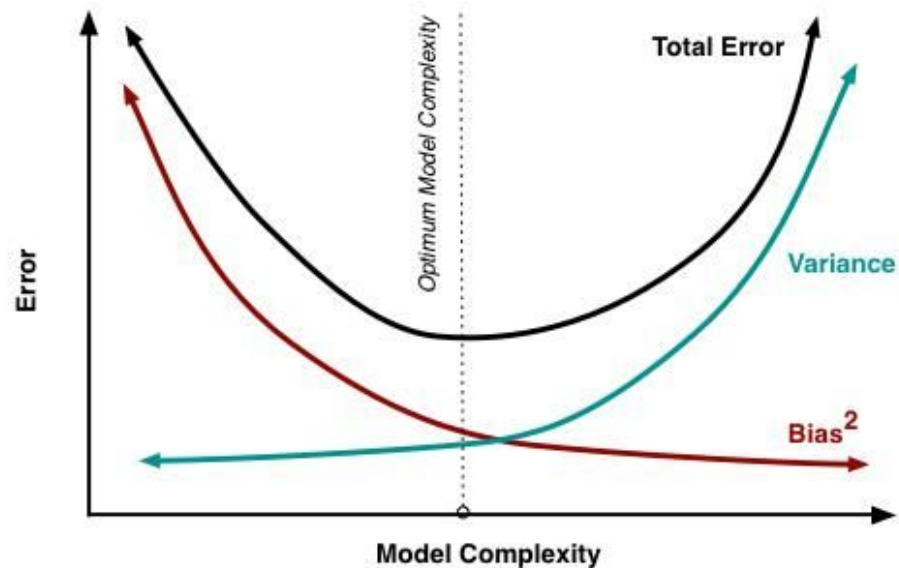
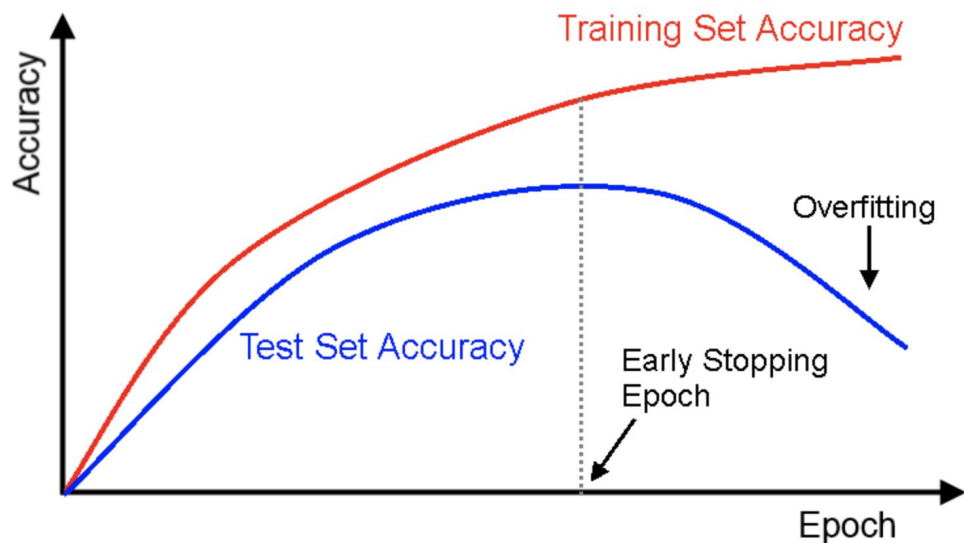
Medidas de regularización: Dropout



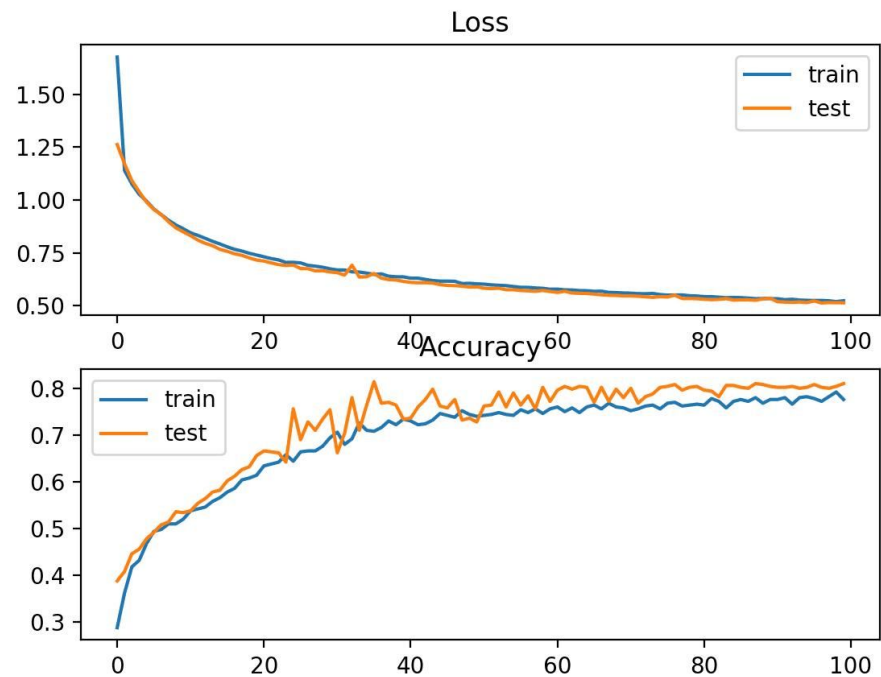
- La red aprende a evitar sobredependencia de ciertas neuronas
- ¡Solo se aplica durante el entrenamiento!



- La red aprende a evitar sobredependencia de ciertas neuronas
- ¡Solo se aplica durante el entrenamiento!



Quando dejamos de aprender?



“He” initialization

$$w_i = \text{Random}(0,1) \sqrt{\frac{2}{n}}$$

$$\text{Var}(w_i) = \frac{2}{n}$$

Usually with:

$$g(z) = \text{ReLU}(z)$$

<http://www.deeplearning.ai/ai-notes/initialization/>

“Xavier” initialization

$$w_i = \text{Random}(0,1) \sqrt{\frac{1}{n}}$$

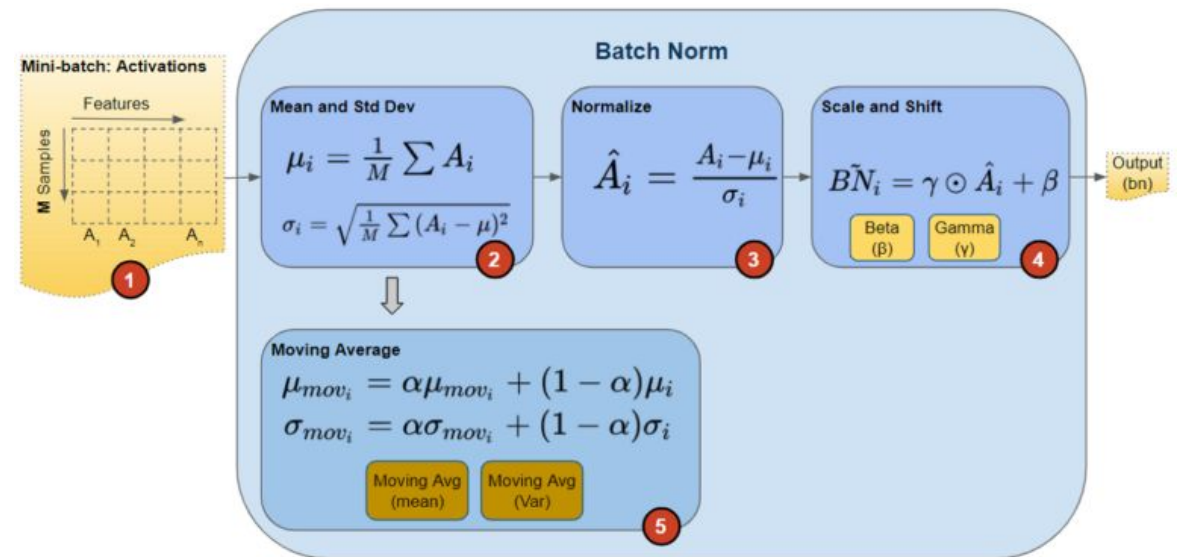
$$\text{Var}(w_i) = \frac{1}{n}$$

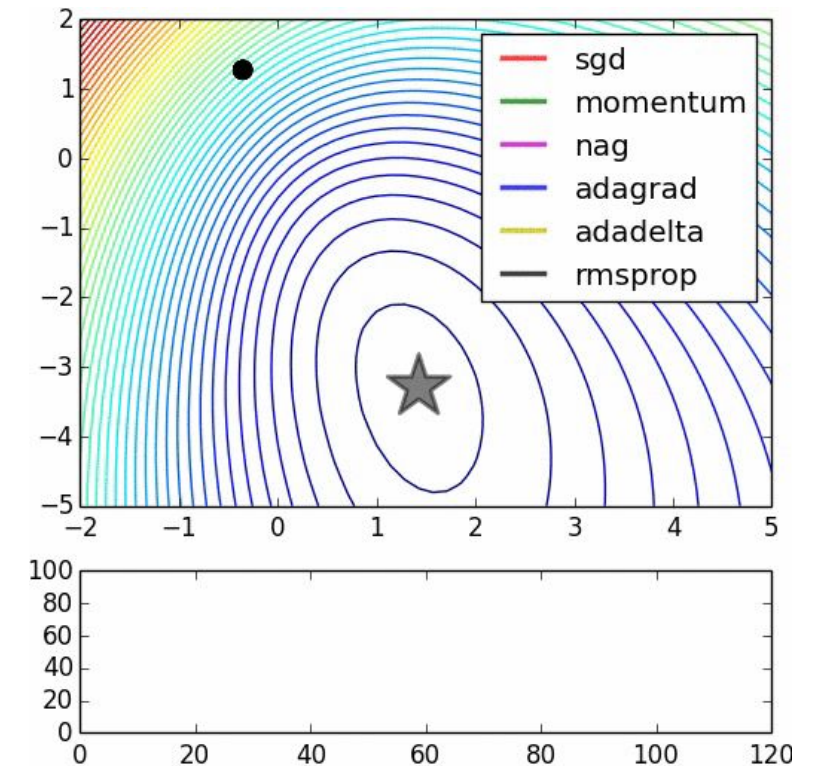
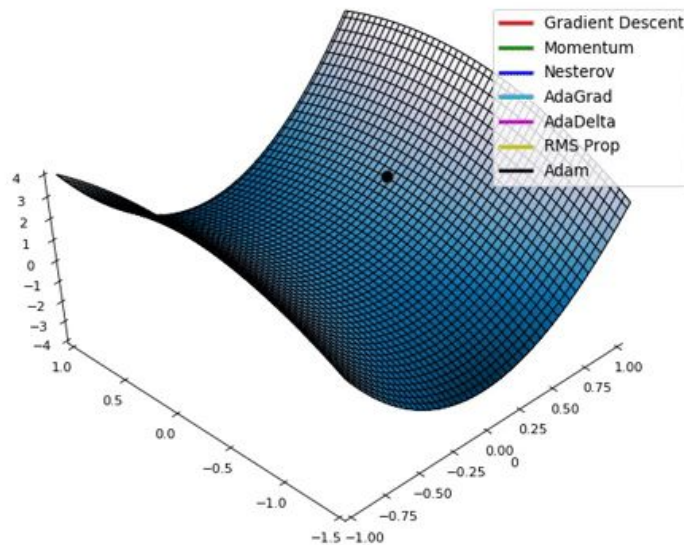
Usually with:

$$g(z) = \tanh(z)$$

Esta capa permite centrar y normalizar cada mini-batch que le llega a la red neuronal con el objetivo de optimizar el entrenamiento

- **Eficiencia en el entrenamiento:** Permite reducir el número de iteraciones de entrenamiento requeridas para alcanzar un cierto nivel de *performance*.
- **Regularización:** Esta técnica introduce cierto “ruido” que puede aprovecharse para mitigar el overfitting
- **Estabilizar el gradiente:** Permite mitigar el desvanecimiento y explosión de gradiente.

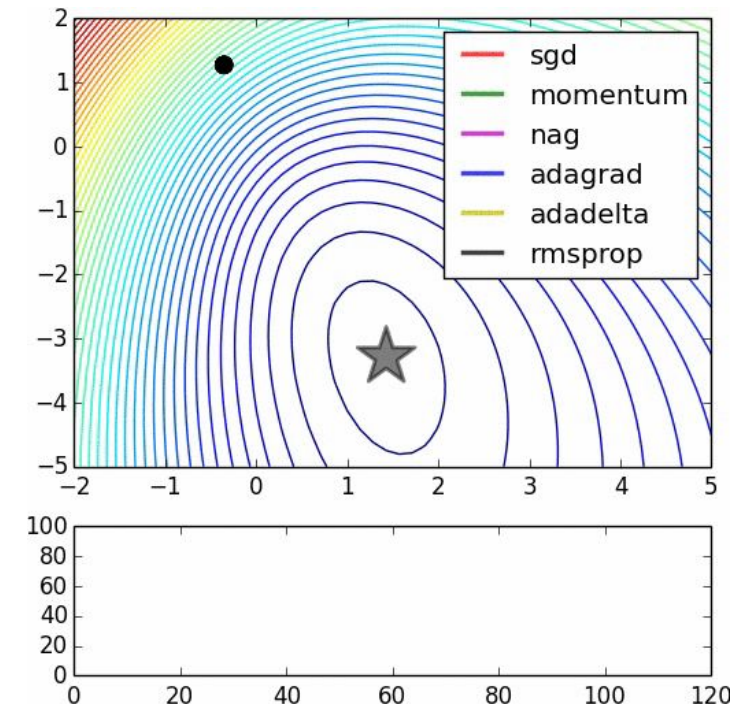
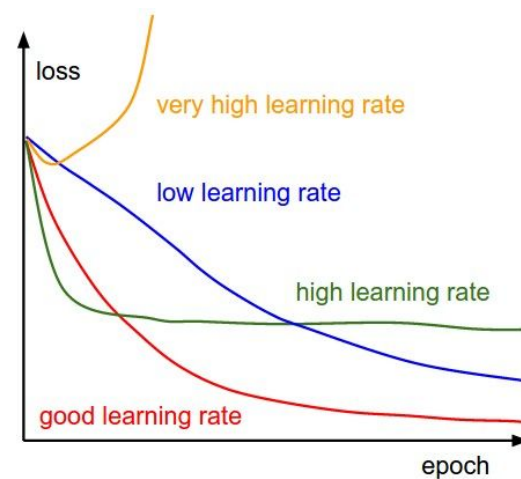
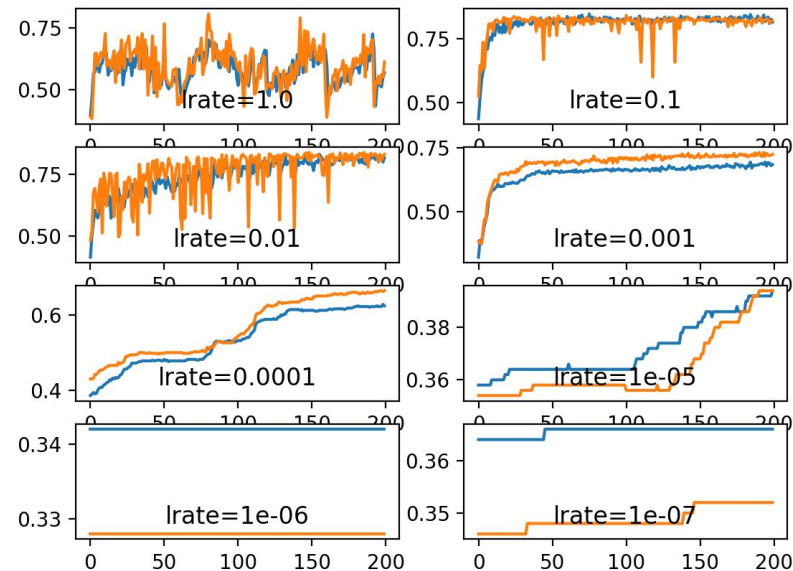




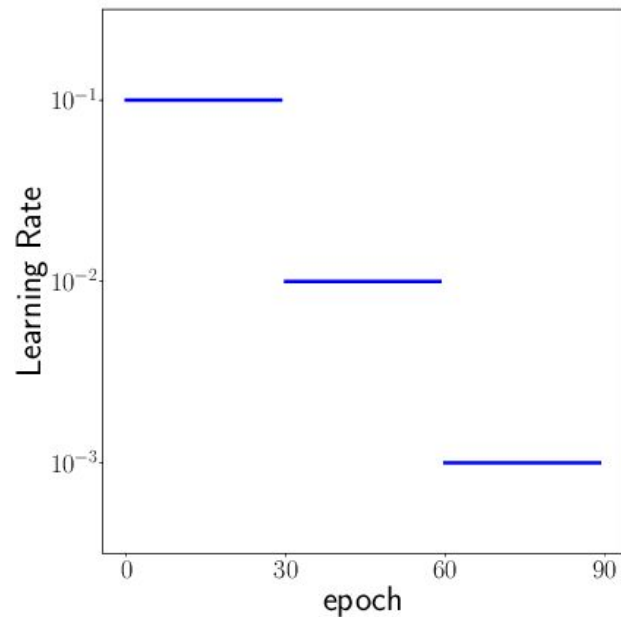
Entrenamiento y técnicas: Learning Rate Decay

Como escojemos la tasa de aprendizaje para que sea la óptima?

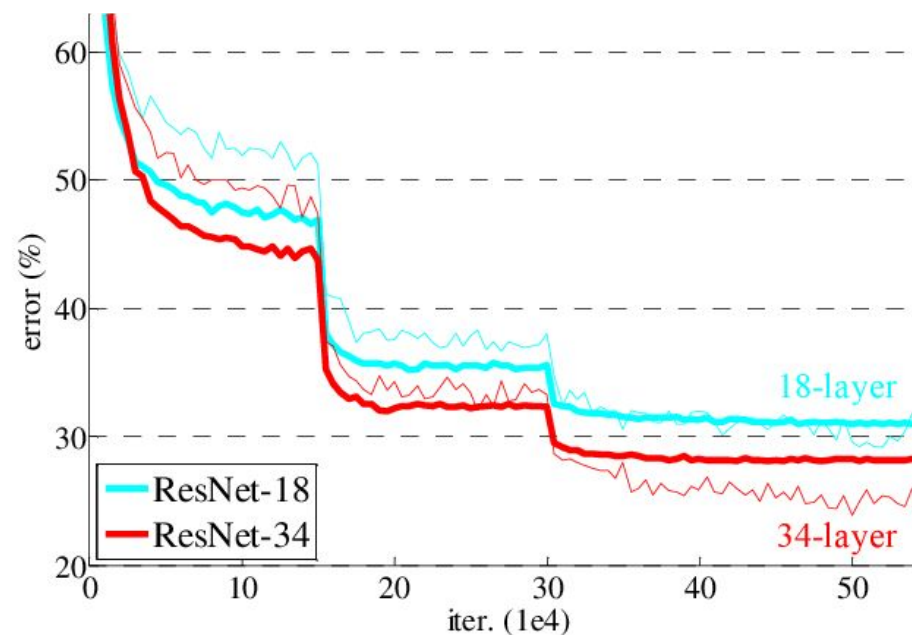
- Y si pudiésemos ir cambiando su valor?



Una manera típica es ir reduciendo el valor del learning rates en órdenes de magnitud en función de la época



(a) Learning rate decay strategy



(b) Figure taken from [He et al. \(2016\)](#)

<https://www.deeplearning.ai/ai-notes/optimization/index.html>

- Tenemos en cuenta los gradientes anteriores para saber cuando “acelerar” y cuando “frenar”
- Nos permite evitar mínimos locales

Regular gradient descent

On iteration k :

1. $m \leftarrow \alpha \partial \theta$
2. $\theta \leftarrow \theta - m$

Iteration 2:

$$\theta_{(2)} = \theta_{(1)} - \alpha \cdot \partial \theta_{(2)}$$

Earlier gradients are ignored

Momentum optimization

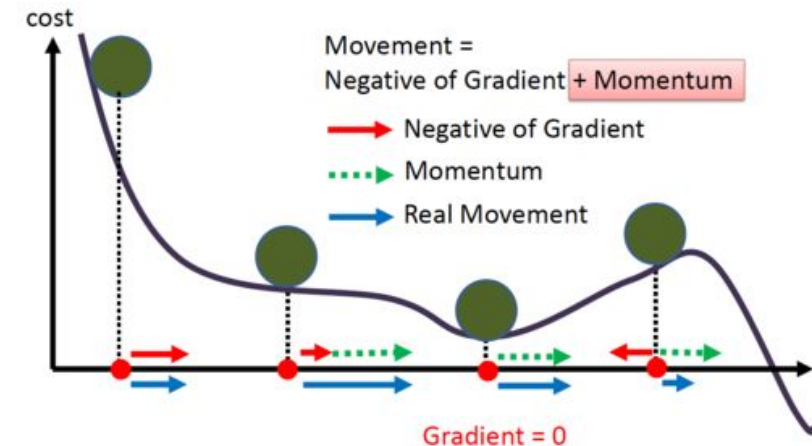
On iteration k :

1. $m \leftarrow \beta m + \alpha \partial \theta$
2. $\theta \leftarrow \theta - m$

Iteration 2:

$$\theta_{(2)} = \theta_{(1)} - \alpha \cdot \partial \theta_{(2)} + \underbrace{\beta(\alpha \cdot \partial \theta_{(1)})}_m$$

Past gradients are accumulated in m

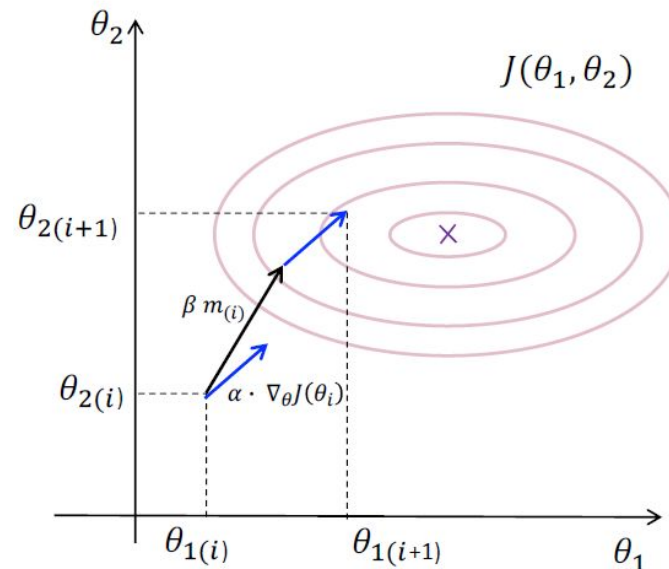


- Usamos el momentum para ver a donde iriamos, y desde ahi es donde calculamos el gradiente para saber a donde movernos
- Intentamos anticipar a donde vamos a ir

Momentum optimization

On iteration k :

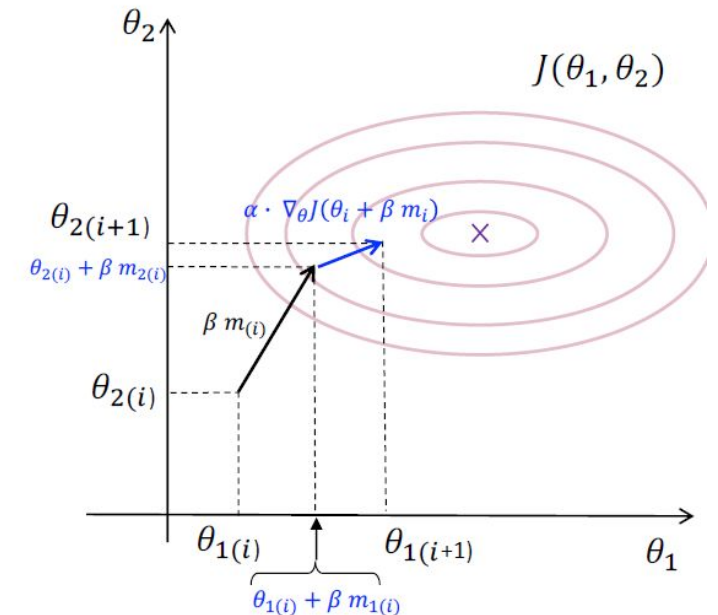
1. $m \leftarrow \beta m + \alpha \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - m$



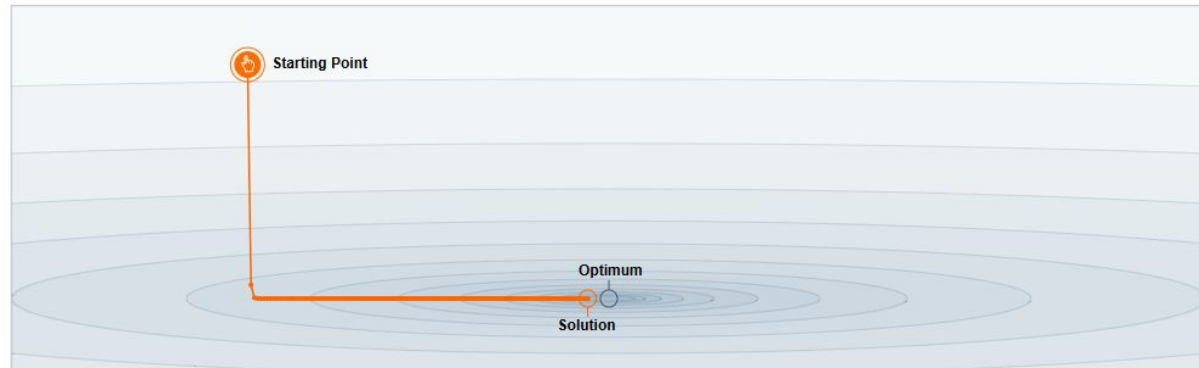
Nesterov accelerated gradient

On iteration k :

1. $m \leftarrow \beta m + \alpha \nabla_{\theta} J(\theta + \beta m)$
2. $\theta \leftarrow \theta - m$



- En función del problema que estamos optimizando, las tasas de aprendizaje necesitan ir ajustándose en función al gradiente
- Podemos usar las direcciones de los gradientes para ir ajustando el tamaño del salto en esa dirección



Regular gradient descent

On iteration k :

$$\begin{aligned} 1. \theta_1 &\leftarrow \theta_1 - \underbrace{\alpha}_{\text{size of step}} \partial \theta_1 \\ &\dots \\ \theta_n &\leftarrow \theta_n - \alpha \partial \theta_n \end{aligned}$$

RSMprop

On iteration k :

$$\begin{aligned} 1. s_1 &\leftarrow \beta s_1 + (1 - \beta) (\partial \theta_1)^2 \\ &\dots \\ 2. \theta_1 &\leftarrow \theta_1 - \underbrace{\alpha \frac{\partial \theta_1}{\sqrt{s_1}}}_{\text{small}} \leftarrow \text{large} \\ &\dots \end{aligned}$$

- Combinamos lo mejor de ambos mundos: momentum y RMSprop
- Implica llevar la cuenta del gradiente medio y su cuadrada
- Ante la duda, Adam

For each Parameter w^j

(j subscript dropped for clarity)

$$\nu_t = \beta_1 * \nu_{t-1} + (1 - \beta_1) * g_t$$

$$s_t = \beta_2 * s_{t-1} + (1 - \beta_2) * g_t^2$$

$$\Delta\omega_t = -\eta \frac{\nu_t}{\sqrt{s_t + \epsilon}} * g_t$$

$$\omega_{t+1} = \omega_t + \Delta\omega_t$$

η : Initial Learning rate

g_t : Gradient at time t along ω^j

ν_t : Exponential Average of gradients along ω_j

s_t : Exponential Average of squares of gradients along ω_j

β_1, β_2 : Hyperparameters





EBIS Business
Techschool

www.ebiseducation.com

